

```
// ===== 文件: xiaozhi/AdminApplication.java =====
package xiaozhi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AdminApplication {

    public static void main(String[] args) {
        SpringApplication.run(AdminApplication.class, args);
        System.out.println("http://localhost:8002/xiaozhi/doc.html");
    }
}

// ===== 文件: xiaozhi/common/config/AliyunOssConfig.java =====
package xiaozhi.common.config;

import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.aliyun.oss.ClientBuilderConfiguration;
import com.aliyun.oss.OSS;
import com.aliyun.oss.OSSClientBuilder;
import com.aliyun.oss.common.auth.DefaultCredentialProvider;
import com.aliyun.oss.common.comm.SignVersion;

import jakarta.annotation.PreDestroy;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

/**
 * 阿里云OSS Client配置类
 */
@Slf4j
@Configuration
@RequiredArgsConstructor
@ConditionalOnProperty(prefix = "aliyun.oss", name = "endpoint", matchIfMissing = false)
public class AliyunOssConfig {

    private final AliyunOssProperties ossProperties;

    private OSS ossClient;

    @Bean
    public OSS ossClient() {
        log.info("初始化阿里云OSS客户端, endpoint={}, bucket={}",
            ossProperties.getEndpoint(), ossProperties.getBucketName());

        ClientBuilderConfiguration config = new ClientBuilderConfiguration();
        config.setSignatureVersion(SignVersion.V4);

        ossClient = OSSClientBuilder.create()
            .endpoint(ossProperties.getEndpoint())
            .credentialsProvider(new DefaultCredentialProvider(
                ossProperties.getAccessKeyId(),
                ossProperties.getAccessKeySecret()))
            .clientConfiguration(config)
            .region(ossProperties.getRegion())
            .build();
    }
}
```

```
        return ossClient;
    }

    @PreDestroy
    public void destroy() {
        if (ossClient != null) {
            ossClient.shutdown();
            log.info("阿里云OSS客户端已关闭");
        }
    }
}

// ===== 文件: xiaozhi/common/config/AliyunOssProperties.java =====
package xiaozhi.common.config;

import org.apache.commons.lang3.StringUtils;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

import lombok.Data;

/**
 * 阿里云OSS配置属性
 */
@Data
@Component
@ConfigurationProperties(prefix = "aliyun.oss")
public class AliyunOssProperties {

    private String endpoint;

    private String accessKeyId;

    private String accessKeySecret;

    private String bucketName;

    private String region;

    /**
     * OSS 公网访问域名，例如 https://oss.eggbase.com。用于把 ossKey 拼接成可访问 URL。
     * 留空时由 OssService 兜底到默认域名，保证向后兼容。
     */
    private String publicUrl;

    /**
     * 判断 OSS 是否已配置完整
     */
    public boolean isConfigured() {
        return StringUtils.isNotBlank(endpoint, accessKeyId, accessKeySecret, bucketName, region);
    }
}

// ===== 文件: xiaozhi/common/config/AsyncConfig.java =====
package xiaozhi.common.config;

import java.util.concurrent.Executor;
import java.util.concurrent.RejectedExecutionHandler;
import java.util.concurrent.ThreadPoolExecutor;

import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

@Configuration
@EnableAsync
@EnableAspectJAutoProxy(exposeProxy = true)
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(4);
        executor.setQueueCapacity(1000);
        executor.setThreadNamePrefix("AsyncThread-");
        // 设置拒绝策略: 由调用线程执行
        executor.setRejectedExecutionHandler(new RejectedExecutionHandler() {
            @Override
            public void rejectedExecution(Runnable r, ThreadPoolExecutor executor) {
                try {
                    // 如果线程池已满, 则由调用线程执行
                    r.run();
                } catch (Exception e) {
                    throw new RuntimeException("执行异步任务失败", e);
                }
            }
        });
        executor.initialize();
        return executor;
    }
}

// ===== 文件: xiaozhi/common/config/MybatisPlusConfig.java =====
package xiaozhi.common.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.BlockAttackInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.OptimisticLockerInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;

import xiaozhi.common.interceptor.DataFilterInterceptor;

/**
 * mybatis-plus配置
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Configuration
public class MybatisPlusConfig {

    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor mybatisPlusInterceptor = new MybatisPlusInterceptor();
        // 数据权限
        mybatisPlusInterceptor.addInnerInterceptor(new DataFilterInterceptor());
        // 分页插件
    }
}
```

```
mybatisPlusInterceptor.addInnerInterceptor(new PaginationInnerInterceptor());
// 乐观锁
mybatisPlusInterceptor.addInnerInterceptor(new OptimisticLockerInnerInterceptor());
// 防止全表更新与删除
mybatisPlusInterceptor.addInnerInterceptor(new BlockAttackInnerInterceptor());

return mybatisPlusInterceptor;
}

}

// ===== 文件: xiaozhi/common/config/RestTemplateConfig.java =====
package xiaozhi.common.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestTemplate;
import java.time.Duration;

/**
 * RestTemplate配置
 */
@Configuration
public class RestTemplateConfig {

    @Bean
    @Primary
    public RestTemplate restTemplate() {
        JdkClientHttpRequestFactory factory = new JdkClientHttpRequestFactory();
        factory.setReadTimeout(Duration.ofSeconds(30));
        return new RestTemplate(factory);
    }
}

// ===== 文件: xiaozhi/common/config/SwaggerConfig.java =====
package xiaozhi.common.config;

import org.springdoc.core.models.GroupedOpenApi;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;

/**
 * Swagger配置
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Configuration
public class SwaggerConfig {

    @Bean
    public GroupedOpenApi deviceApi() {
        return GroupedOpenApi.builder()
            .group("device")
            .pathsToMatch("/device/**")
            .build();
    }
}
```

```
@Bean
public GroupedOpenApi agentApi() {
    return GroupedOpenApi.builder()
        .group("agent")
        .pathsToMatch("/agent/**")
        .build();
}

@Bean
public GroupedOpenApi modelApi() {
    return GroupedOpenApi.builder()
        .group("models")
        .pathsToMatch("/models/**")
        .build();
}

@Bean
public GroupedOpenApi oatApi() {
    return GroupedOpenApi.builder()
        .group("ota")
        .pathsToMatch("/ota/**", "/otaMag/**")
        .build();
}

@Bean
public GroupedOpenApi timbreApi() {
    return GroupedOpenApi.builder()
        .group("timbre")
        .pathsToMatch("/ttsVoice/**")
        .build();
}

@Bean
public GroupedOpenApi sysApi() {
    return GroupedOpenApi.builder()
        .group("admin")
        .pathsToMatch("/admin/**")
        .build();
}

@Bean
public GroupedOpenApi userApi() {
    return GroupedOpenApi.builder()
        .group("user")
        .pathsToMatch("/user/**")
        .build();
}

@Bean
public GroupedOpenApi configApi() {
    return GroupedOpenApi.builder()
        .group("config")
        .pathsToMatch("/config/**")
        .build();
}

@Bean
public GroupedOpenApi knowledgeApi() {
    return GroupedOpenApi.builder()
        .group("knowledge")
        .build();
}
```

```

        .pathsToMatch("/datasets/**")
        .build();
    }

    @Bean
    public GroupedOpenApi botApi() {
        return GroupedOpenApi.builder()
            .group("bot")
            .pathsToMatch("/api/v1/**")
            .build();
    }

    @Bean
    public GroupedOpenApi correctWordApi() {
        return GroupedOpenApi.builder()
            .group("correct-word")
            .pathsToMatch("/correct-word/**")
            .build();
    }

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI().info(new Info()
            .title("xiaozi-esp32-manager-api")
            .description("xiaozi-esp32-manager-api文档")
            .version("3.0")
            .termsOfService("https://127.0.0.1"));
    }
}

// ===== 文件: xiaozi/common/exception/ErrorCode.java =====
package xiaozi.common.exception;

/**
 * 错误编码，由5位数字组成，前2位为模块编码，后3位为业务编码
 * <p>
 * 如：10001（10代表系统模块，001代表业务代码）
 * </p>
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
public interface ErrorCode {
    int INTERNAL_SERVER_ERROR = 500;
    int UNAUTHORIZED = 401;
    int FORBIDDEN = 403;

    int NOT_NULL = 10001;
    int DB_RECORD_EXISTS = 10002;
    int PARAMS_GET_ERROR = 10003;
    int ACCOUNT_PASSWORD_ERROR = 10004;
    int ACCOUNT_DISABLE = 10005;
    int IDENTIFIER_NOT_NULL = 10006;
    int CAPTCHA_ERROR = 10007;
    int PHONE_NOT_NULL = 10008;
    int PASSWORD_ERROR = 10009;

    int SUPERIOR_DEPT_ERROR = 10011;
    int SUPERIOR_MENU_ERROR = 10012;
    int DATA_SCOPE_PARAMS_ERROR = 10013;
    int DEPT_SUB_DELETE_ERROR = 10014;
    int DEPT_USER_DELETE_ERROR = 10015;
}

```

```
int UPLOAD_FILE_EMPTY = 10019;
int TOKEN_NOT_EMPTY = 10020;
int TOKEN_INVALID = 10021;
int ACCOUNT_LOCK = 10022;

int OSS_UPLOAD_FILE_ERROR = 10024;

int REDIS_ERROR = 10027;
int JOB_ERROR = 10028;
int INVALID_SYMBOL = 10029;
int PASSWORD_LENGTH_ERROR = 10030;
int PASSWORD_WEAK_ERROR = 10031;
int DEL_MYSELF_ERROR = 10032;
int DEVICE_CAPTCHA_ERROR = 10033;

// 参数校验相关错误码
int PARAM_VALUE_NULL = 10034;
int PARAM_TYPE_NULL = 10035;
int PARAM_TYPE_INVALID = 10036;
int PARAM_NUMBER_INVALID = 10037;
int PARAM_BOOLEAN_INVALID = 10038;
int PARAM_ARRAY_INVALID = 10039;
int PARAM_JSON_INVALID = 10040;

int OTA_DEVICE_NOT_FOUND = 10041;
int OTA_DEVICE_NEED_BIND = 10042;

// 新增错误编码
int DELETE_DATA_FAILED = 10043;
int USER_NOT_LOGIN = 10044;
int WEB_SOCKET_CONNECT_FAILED = 10045;
int VOICE_PRINT_SAVE_ERROR = 10046;
int TODAY_SMS_LIMIT_REACHED = 10047;
int OLD_PASSWORD_ERROR = 10048;
int INVALID_LLM_TYPE = 10049;
int TOKEN_GENERATE_ERROR = 10050;
int RESOURCE_NOT_FOUND = 10051;

// 新增错误编码
int DEFAULT_AGENT_NOT_FOUND = 10052;
int AGENT_NOT_FOUND = 10053;
int VOICEPRINT_API_NOT_CONFIGURED = 10054;
int SMS_SEND_FAILED = 10055;
int SMS_CONNECTION_FAILED = 10056;
int AGENT_VOICEPRINT_CREATE_FAILED = 10057;
int AGENT_VOICEPRINT_UPDATE_FAILED = 10058;
int AGENT_VOICEPRINT_DELETE_FAILED = 10059;
int SMS_SEND_TOO_FREQUENTLY = 10060;
int ACTIVATION_CODE_EMPTY = 10061;
int ACTIVATION_CODE_ERROR = 10062;
int DEVICE_ALREADY_ACTIVATED = 10063;
// 默认模型删除错误
int DEFAULT_MODEL_DELETE_ERROR = 10064;
// 登录相关错误码
int ADD_DATA_FAILED = 10065; // 新增数据失败
int UPDATE_DATA_FAILED = 10066; // 修改数据失败
int SMS_CAPTCHA_ERROR = 10067; // 短信验证码错误
int MOBILE_REGISTER_DISABLED = 10068; // 未开启手机注册
int USERNAME_NOT_PHONE = 10069; // 用户名不是手机号码
```

```
int PHONE_ALREADY_REGISTERED = 10070; // 手机号码已注册
int PHONE_NOT_REGISTERED = 10071; // 手机号码未注册
int USER_REGISTER_DISABLED = 10072; // 不允许用户注册
int RETRIEVE_PASSWORD_DISABLED = 10073; // 未开启找回密码功能
int PHONE_FORMAT_ERROR = 10074; // 手机号码格式不正确
int SMS_CODE_ERROR = 10075; // 手机验证码错误

// 字典类型相关错误码
int DICT_TYPE_NOT_EXIST = 10076; // 字典类型不存在
int DICT_TYPE_DUPLICATE = 10077; // 字典类型编码重复

// 资源处理相关错误码
int RESOURCE_READ_ERROR = 10078; // 读取资源失败

// 邮件相关错误码
int EMAIL_SEND_FAILED = 10402; // 邮件发送失败
int EMAIL_CONNECTION_FAILED = 10403; // 邮件连接建立失败
int EMAIL_FORMAT_ERROR = 10404; // 邮箱格式不正确

// 智能体相关错误码
int LLM_INTENT_PARAMS_MISMATCH = 10079; // LLM大模型和Intent意图识别，选择参数不匹配

// 声纹相关错误码
int VOICEPRINT_ALREADY_REGISTERED = 10080; // 此声音声纹已经注册
int VOICEPRINT_DELETE_ERROR = 10081; // 删除声纹出现错误
int VOICEPRINT_UPDATE_NOT_ALLOWED = 10082; // 声纹修改不允许，声音已注册
int VOICEPRINT_UPDATE_ADMIN_ERROR = 10083; // 修改声纹错误，请联系管理员
int VOICEPRINT_API_URI_ERROR = 10084; // 声纹接口地址错误
int VOICEPRINT_AUDIO_NOT_BELONG_AGENT = 10085; // 音频数据不属于智能体
int VOICEPRINT_AUDIO_EMPTY = 10086; // 音频数据为空
int VOICEPRINT_REGISTER_REQUEST_ERROR = 10087; // 声纹保存请求失败
int VOICEPRINT_REGISTER_PROCESS_ERROR = 10088; // 声纹保存处理失败
int VOICEPRINT_UNREGISTER_REQUEST_ERROR = 10089; // 声纹注销请求失败
int VOICEPRINT_UNREGISTER_PROCESS_ERROR = 10090; // 声纹注销处理失败
int VOICEPRINT_IDENTIFY_REQUEST_ERROR = 10091; // 声纹识别请求失败

int LLM_NOT_EXIST = 10092; // 设置的LLM不存在
int MODEL_REFERENCED_BY_AGENT = 10093; // 该模型配置已被智能体引用，无法删除
int LLM_REFERENCED_BY_INTENT = 10094; // 该LLM模型已被意图识别配置引用，无法删除

// 服务端管理相关错误码
int INVALID_SERVER_ACTION = 10095; // 无效服务端操作
int SERVER_WEBSOCKET_NOT_CONFIGURED = 10096; // 未配置服务端WebSocket地址
int TARGET_WEBSOCKET_NOT_EXIST = 10097; // 目标WebSocket地址不存在

// 参数验证相关错误码
int WEBSOCKET_URLS_EMPTY = 10098; // WebSocket地址列表不能为空
int WEBSOCKET_URL_LOCALHOST = 10099; // WebSocket地址不能使用localhost或127.0.0.1
int WEBSOCKET_URL_FORMAT_ERROR = 10100; // WebSocket地址格式不正确
int WEBSOCKET_CONNECTION_FAILED = 10101; // WebSocket连接测试失败
int OTA_URL_EMPTY = 10102; // OTA地址不能为空
int OTA_URL_LOCALHOST = 10103; // OTA地址不能使用localhost或127.0.0.1
int OTA_URL_PROTOCOL_ERROR = 10104; // OTA地址必须以http或https开头
int OTA_URL_FORMAT_ERROR = 10105; // OTA地址必须以/ota/结尾
int OTA_INTERFACE_ACCESS_FAILED = 10106; // OTA接口访问失败
int OTA_INTERFACE_FORMAT_ERROR = 10107; // OTA接口返回内容格式不正确
int OTA_INTERFACE_VALIDATION_FAILED = 10108; // OTA接口验证失败
int MCP_URL_EMPTY = 10109; // MCP地址不能为空
int MCP_URL_LOCALHOST = 10110; // MCP地址不能使用localhost或127.0.0.1
int MCP_URL_INVALID = 10111; // 不是正确的MCP地址
```

```
int MCP_INTERFACE_ACCESS_FAILED = 10112; // MCP接口访问失败
int MCP_INTERFACE_FORMAT_ERROR = 10113; // MCP接口返回内容格式不正确
int MCP_INTERFACE_VALIDATION_FAILED = 10114; // MCP接口验证失败
int VOICEPRINT_URL_EMPTY = 10115; // 声纹接口地址不能为空
int VOICEPRINT_URL_LOCALHOST = 10116; // 声纹接口地址不能使用localhost或127.0.0.1
int VOICEPRINT_URL_INVALID = 10117; // 不是正确的声纹接口地址
int VOICEPRINT_URL_PROTOCOL_ERROR = 10118; // 声纹接口地址必须以http或https开头
int VOICEPRINT_INTERFACE_ACCESS_FAILED = 10119; // 声纹接口访问失败
int VOICEPRINT_INTERFACE_FORMAT_ERROR = 10120; // 声纹接口返回内容格式不正确
int VOICEPRINT_INTERFACE_VALIDATION_FAILED = 10121; // 声纹接口验证失败
int MQTT_SECRET_EMPTY = 10122; // mqtt密钥不能为空
int MQTT_SECRET_LENGTH_INSECURE = 10123; // mqtt密钥长度不安全
int MQTT_SECRET_CHARACTER_INSECURE = 10124; // mqtt密钥必须同时包含大小写字母
int MQTT_SECRET_WEAK_PASSWORD = 10125; // mqtt密钥包含弱密码
int DICT_LABEL_DUPLICATE = 10128; // 字典标签重复
int SM2_KEY_NOT_CONFIGURED = 10129; // SM2密钥未配置
int SM2_DECRYPT_ERROR = 10130; // SM2解密失败
int MODEL_TYPE_PROVIDE_CODE_NOT_NULL = 10131; // modelType和provideCode不能为空

// 聊天记录相关错误码
int CHAT_HISTORY_NO_PERMISSION = 10132; // 没有权限查看该智能体的聊天记录
int CHAT_HISTORY_SESSION_ID_NOT_NULL = 10133; // 会话ID不能为空
int CHAT_HISTORY_AGENT_ID_NOT_NULL = 10134; // 智能体ID不能为空
int CHAT_HISTORY_DOWNLOAD_FAILED = 10135; // 聊天记录下载失败
int DOWNLOAD_LINK_EXPIRED = 10136; // 下载链接已过期或无效
int DOWNLOAD_LINK_INVALID = 10137; // 下载链接无效
int CHAT_ROLE_USER = 10138; // 用户角色
int CHAT_ROLE_AGENT = 10139; // 智能体角色

// 声音克隆相关错误码
int VOICE_CLONE_AUDIO_EMPTY = 10140; // 音频文件不能为空
int VOICE_CLONE_NOT_AUDIO_FILE = 10141; // 只支持音频文件
int VOICE_CLONE_AUDIO_TOO_LARGE = 10142; // 音频文件大小不能超过10MB
int VOICE_CLONE_UPLOAD_FAILED = 10143; // 上传失败
int VOICE_CLONE_RECORD_NOT_EXIST = 10144; // 声音克隆记录不存在
int VOICE_RESOURCE_INFO_EMPTY = 10145; // 音色资源信息不能为空
int VOICE_RESOURCE_PLATFORM_NAME_EMPTY = 10146; // 平台名称不能为空
int VOICE_RESOURCE_ID_EMPTY = 10147; // 音色ID不能为空
int VOICE_RESOURCE_ACCOUNT_EMPTY = 10148; // 归属账号不能为空
int VOICE_RESOURCE_DELETE_ID_EMPTY = 10149; // 删除的音色资源ID不能为空
int VOICE_RESOURCE_NO_PERMISSION = 10150; // 您没有权限操作该记录
int VOICE_CLONE_AUDIO_NOT_UPLOADED = 10151; // 请先上传音频文件
int VOICE_CLONE_MODEL_CONFIG_NOT_FOUND = 10152; // 模型配置未找到
int VOICE_CLONE_MODEL_TYPE_NOT_FOUND = 10153; // 模型类型未找到
int VOICE_CLONE_TRAINING_FAILED = 10154; // 训练失败
int VOICE_CLONE_HUOSHAN_CONFIG_MISSING = 10155; // 火山引擎缺少配置
int VOICE_CLONE_RESPONSE_FORMAT_ERROR = 10156; // 响应格式错误
int VOICE_CLONE_REQUEST_FAILED = 10157; // 请求失败
int VOICE_CLONE_PREFIX = 10158; // 复刻音色前缀
int VOICE_ID_ALREADY_EXISTS = 10159; // 音色ID已存在
int VOICE_CLONE_HUOSHAN_VOICE_ID_ERROR = 10160; // 火山引擎音色ID格式错误

// 设备相关错误码
int MAC_ADDRESS_ALREADY_EXISTS = 10161; // Mac地址已存在

// 模型相关错误码
int MODEL_PROVIDER_NOT_EXIST = 10162; // 供应器不存在

// 知识库相关错误码
int Knowledge_Base_RECORD_NOT_EXISTS = 10163; // 知识库记录不存在
int RAG_CONFIG_NOT_FOUND = 10164; // RAG配置未找到
```

```
int RAG_CONFIG_TYPE_ERROR = 10165; // RAG配置类型错误
int RAG_DEFAULT_CONFIG_NOT_FOUND = 10166; // 默认RAG配置未找到
int RAG_API_ERROR = 10167; // RAG调用失败
int UPLOAD_FILE_ERROR = 10168; // 上传文件失败
int NO_PERMISSION = 10169; // 没有权限
int KNOWLEDGE_BASE_NAME_EXISTS = 10170; // 同名知识库已存在
int RAG_API_ERROR_URL_NULL = 10171; // RAG配置中base_url为空, 请完善配置
int RAG_API_ERROR_API_KEY_NULL = 10172; // RAG配置中api_key为空, 请完善配置
int RAG_API_ERROR_API_KEY_INVALID = 10173; // RAG配置中api_key包含占位符, 请替换为实际的API密钥
int RAG_API_ERROR_URL_INVALID = 10174; // RAG配置中base_url格式不正确, 请检查协议是否正确
int RAG_DATASET_ID_NOT_NULL = 10176; // RAG配置中dataset_id不能为空
int RAG_MODEL_ID_NOT_NULL = 10177; // RAG配置中model_id不能为空
int RAG_DATASET_ID_AND_MODEL_ID_NOT_NULL = 10178; // RAG配置中dataset_id和model_id不能为空
int RAG_FILE_NAME_NOT_NULL = 10179; // 文件名称不能为空
int RAG_FILE_CONTENT_EMPTY = 10180; // 文件内容不能为空

// 设备相关错误码(补充)
int MCA_NOT_NULL = 10175; // mac地址不能为空

// 音色克隆(补充)
int VOICE_CLONE_NAME_NOT_NULL = 10181; // 音色克隆名称不能为空
int VOICE_CLONE_AUDIO_NOT_FOUND = 10182; // 音色克隆音频不存在

// 智能体模板相关错误码(补充)
int AGENT_TEMPLATE_NOT_FOUND = 10183; // 默认智能体未找到

// 知识库适配器相关错误码
int RAG_ADAPTER_TYPE_NOT_SUPPORTED = 10184; // 不支持的适配器类型
int RAG_CONFIG_VALIDATION_FAILED = 10185; // RAG配置验证失败
int RAG_ADAPTER_CREATION_FAILED = 10186; // 适配器创建失败
int RAG_ADAPTER_INIT_FAILED = 10187; // 适配器初始化失败
int RAG_ADAPTER_CONNECTION_FAILED = 10188; // 适配器连接测试失败
int RAG_ADAPTER_OPERATION_FAILED = 10189; // 适配器操作失败
int RAG_ADAPTER_NOT_FOUND = 10190; // 适配器未找到
int RAG_ADAPTER_CACHE_ERROR = 10191; // 适配器缓存错误
int RAG_ADAPTER_TYPE_NOT_FOUND = 10192; // 适配器类型未找到

// 设备工具相关错误码
int DEVICE_ID_NOT_NULL = 10193; // 设备ID不能为空
int DEVICE_NOT_EXIST = 10194; // 设备不存在
int OTA_UPLOAD_COUNT_EXCEED = 10195; // OTA上传次数超过限制

// 智能体标签相关错误码
int AGENT_TAG_NAME_DUPLICATE = 10196; // 标签名称已存在
int AGENT_TAG_NAME_EMPTY = 10197; // 标签名称不能为空
int AGENT_TAG_NOT_EXIST = 10198; // 标签不存在

int RAG_DOCUMENT_PARSING_DELETE_ERROR = 10199; // 文档解析中, 禁止删除

// 智能体MCP相关错误码
int MCP_ACCESS_POINT_ADDRESS_NO_PERMISSION = 10200; // 没有权限查看该智能体的MCP接入点地址
int MCP_ACCESS_POINT_ADDRESS_NOT_CONFIGURED = 10201; // 请联系管理员进入参数管理配置mcp接入点地址
int MCP_ACCESS_POINT_TOOLS_LIST_NO_PERMISSION = 10202; // 没有权限查看该智能体的MCP工具列表

// 替换词相关错误码
int CORRECT_WORD_FILE_NAME_EXISTS = 10203; // 文件名已存在
int FILE_SIZE_OVER_LIMIT = 10204; // 文件大小超过限制

// AI宠物相关错误码
int PET_DEVICE_NOT_FOUND = 10205; // 宠物关联的设备不存在
```

```
int PET_ALREADY_EXISTS = 10206; // 该账号已创建过宠物
int PET_NOT_FOUND = 10207; // 宠物不存在
int PET_NO_PERMISSION = 10208; // 没有权限操作该宠物
int PET_ALREADY_HATCHED = 10209; // 已破壳，不能再做修炼动作
int PET_HATCH_TIME_NOT_REACHED = 10214; // 还未到破壳时间，请耐心等待
int PET_COLLECTION_CARD_LIMIT_REACHED = 10222; // 收藏卡已集齐(10/10)

// AI伴侣相关错误码
int COMPANION_NOT_FOUND = 10210; // 伴侣不存在
int COMPANION_ALREADY_EXISTS = 10211; // 该设备已创建过伴侣
int COMPANION_INVALID_CHARACTER = 10212; // 无效的角色编码

// 支付相关错误码
int PAY_CHANNEL_NOT_AVAILABLE = 10300; // 支付渠道不可用
int PAY_ORDER_NOT_FOUND = 10301; // 订单不存在
int PAY_ORDER_AMOUNT_MISMATCH = 10302; // 订单金额不一致
int PAY_ORDER_DUPLICATE = 10303; // 重复下单，请稍后再试
int PAY_SIGN_INVALID = 10304; // 支付签名校验失败
int PAY_REFUND_FAILED = 10305; // 退款失败
int PAY_ORDER_STATUS_INVALID = 10306; // 订单状态不允许该操作
int PAY_PRODUCT_INVALID = 10307; // 商品信息无效
int PAY_OPENID_REQUIRED = 10308; // 缺少微信openid，无法发起支付

// 订阅相关错误码
int SUBSCRIPTION_PLAN_NOT_FOUND = 10310; // 订阅档位不存在
int SUBSCRIPTION_NOT_ACTIVE = 10311; // 当前没有生效的订阅
int SUBSCRIPTION_FEATURE_DENIED = 10312; // 当前订阅未包含该能力
int SUBSCRIPTION_DOWNGRADE_DENIED = 10313; // 不允许降级订阅

// 道具相关错误码
int ITEM_SKU_NOT_FOUND = 10320; // 道具不存在
int ITEM_INSUFFICIENT = 10321; // 道具数量不足
int ITEM_CONSUME_FAILED = 10322; // 道具消耗失败

// OSS相关错误码
int OSS_DOWNLOAD_FILE_ERROR = 10400; // OSS下载文件失败
int OSS_DELETE_FILE_ERROR = 10401; // OSS删除文件失败

// 用户资料相关错误码
int NICKNAME_TOO_LONG = 10215; // 昵称最多16个字符
int NICKNAME_SENSITIVE = 10216; // 昵称含有不适合的内容
int INVALID_GENDER = 10217; // 性别格式错误
int INVALID_MBTI = 10218; // MBTI类型错误
int INVALID_BIRTHDAY = 10219; // 生日格式错误
int AVATAR_FILE_TYPE_ERROR = 10220; // 头像文件类型错误，仅支持jpg/png/webp
int CITY_TOO_LONG = 10221; // 城市最多32个字符
int IMAGE_FILE_TYPE_ERROR = 10222; // 图片文件类型错误
int INVALID_AGE_RANGE = 10223; // 年龄区间错误

// 用户反馈相关错误码
int FEEDBACK_SUBMIT_TOO_FREQUENTLY = 10330; // 反馈提交过于频繁，请稍后再试
int FEEDBACK_NOT_FOUND = 10331; // 反馈记录不存在

// PDC NFC 实物生产域错误码
int PDC_NFC_FEATURE_DISABLED = 10500; // NFC 功能未开启
int PDC_NFC_MODEL_ID_NOT_CONFIGURED = 10501; // NFC 模型ID未配置
int PDC_NFC_RELEASE_NOT_READY = 10502; // NFC 发布就绪开关未开 (release-ready)
int PDC_NFC_CRYPTONOT_CONFIGURED = 10503; // NFC 密钥未配置
int PDC_NFC_INVALID_STATE = 10504; // NFC 状态转换不合法
int PDC_NFC_ASSET_NOT_FOUND = 10505; // NFC 资产不存在
```

```
int PDC_NFC_ASSET_UNAVAILABLE = 10506; // NFC 资产不可用
int PDC_NFC_ASSET_ALREADY_CLAIMED = 10507; // NFC 资产已被领取
int PDC_NFC_IDEMPOTENCY_CONFLICT = 10508; // NFC 幂等冲突
int PDC_NFC_BATCH_NOT_FOUND = 10509; // NFC 批次不存在
int PDC_NFC_JOB_NOT_FOUND = 10510; // NFC 任务不存在
int PDC_NFC_JOB_CONFLICT = 10511; // NFC 任务冲突
int PDC_NFC_WECHAT_NFC_ERROR = 10512; // 微信 NFC 接口错误
int PDC_NFC_CSV_FORMAT_ERROR = 10513; // CSV 格式错误
int PDC_NFC_CSV_CONTENT_MISMATCH = 10514; // CSV 内容不匹配
int PDC_NFC_BULK_LIMIT_EXCEEDED = 10515; // 批量操作超限
int PDC_NFC_PHONE_REQUIRED = 10516; // 需要手机号
int PDC_NFC_RATE_LIMITED = 10517; // 操作频率受限
int PDC_NFC_INVALID_PROTOTYPE = 10518; // 无效的原型编码
int PDC_NFC_INVALID_MODEL_ID = 10519; // 无效的模型ID
int PDC_NFC_WRITE_RESULT_CONFLICT = 10520; // 写卡结果冲突
int PDC_NFC_RELEASE_EVIDENCE_MISSING = 10521; // 缺少当前版本的发布证据（需在商品类型管理登记）
int PDC_NFC_NO_AVAILABLE_ASSETS = 10522; // 批次内没有可处理状态的资产
int PDC_NFC_ASSET_DATA_INCONSISTENT = 10523; // 资产数据不一致（记录缺失或 Scheme 加密字段不完整）
int PDC_NFC_INVALID_JOB_MODE = 10524; // 写卡任务模式无效
int PDC_NFC_JOB_MODE_MISMATCH = 10525; // 操作与写卡任务模式不符（工厂CSV/手动通道互斥）
int PDC_NFC_ASSET_NOT_LOCKED = 10526; // 手动模式资产未锁卡，禁止入库
int PDC_NFC_LOCK_NOT_VERIFIED = 10527; // 手动模式资产锁后未完成触碰复验，禁止入库
}

// ===== 文件: xiaozhi/common/exception/RenException.java =====
package xiaozhi.common.exception;

import xiaozhi.common.utils.MessageUtils;

/**
 * 自定义异常
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
public class RenException extends RuntimeException {

    private int code;
    private String msg;

    public RenException(int code) {
        super(MessageUtils.getMessage(code));
        this.code = code;
        this.msg = MessageUtils.getMessage(code);
    }

    public RenException(int code, String... params) {
        super(MessageUtils.getMessage(code, params));
        this.code = code;
        this.msg = MessageUtils.getMessage(code, params);
    }

    public RenException(int code, Throwable e) {
        super(MessageUtils.getMessage(code), e);
        this.code = code;
        this.msg = MessageUtils.getMessage(code);
    }

    public RenException(int code, Throwable e, String... params) {
        super(MessageUtils.getMessage(code, params), e);
        this.code = code;
        this.msg = MessageUtils.getMessage(code, params);
    }
}
```

```
}

public RenException(String msg) {
    super(msg);
    this.code = ErrorCode.INTERNAL_SERVER_ERROR;
    this.msg = msg;
}

public RenException(String msg, Throwable e) {
    super(msg, e);
    this.code = ErrorCode.INTERNAL_SERVER_ERROR;
    this.msg = msg;
}

public String getMsg() {
    return msg;
}

public void setMsg(String msg) {
    this.msg = msg;
}

public int getCode() {
    return code;
}

public void setCode(int code) {
    this.code = code;
}
}

// ===== 文件: xiaozhi/common/exception/RenExceptionHandler.java =====
package xiaozhi.common.exception;

import java.util.List;
import java.util.Objects;

import org.apache.shiro.authz.UnauthorizedException;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.validation.ObjectError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.resource.NoResourceFoundException;

import lombok.AllArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import xiaozhi.common.utils.MessageUtils;
import xiaozhi.common.utils.Result;

/**
 * 异常处理器
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Slf4j
@AllArgsConstructor
@RestControllerAdvice
public class RenExceptionHandler {
```

```
/**
 * 处理自定义异常
 */
@ExceptionHandler(RenException.class)
public Result<Void> handleRenException(RenException ex) {
    Result<Void> result = new Result<>();
    result.error(ex.getCode(), ex.getMsg());

    return result;
}

@ExceptionHandler(DuplicateKeyException.class)
public Result<Void> handleDuplicateKeyException(DuplicateKeyException ex) {
    Result<Void> result = new Result<>();
    result.error(ErrorCode.DB_RECORD_EXISTS);

    return result;
}

@ExceptionHandler(UnauthorizedException.class)
public Result<Void> handleUnauthorizedException(UnauthorizedException ex) {
    Result<Void> result = new Result<>();
    result.error(ErrorCode.FORBIDDEN);

    return result;
}

@ExceptionHandler(Exception.class)
public Result<Void> handleException(Exception ex) {
    log.error(ex.getMessage(), ex);

    return new Result<Void>().error();
}

@ExceptionHandler(NoResourceFoundException.class)
public Result<Void> handleNoResourceFoundException(NoResourceFoundException ex) {
    log.warn("Resource not found: {}", ex.getMessage());
    return new Result<Void>().error(404, MessageUtils.getMessage(ErrorCode.RESOURCE_NOT_FOUND));
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public Result<Void> handleMethodArgumentNotValidException(MethodArgumentNotValidException ex) {
    List<ObjectError> allErrors = ex.getBindingResult().getAllErrors();
    String errorMsg = allErrors.stream()
        .filter(Objects::nonNull)
        .map(err -> {
            String msg = err.getDefaultMessage();
            return (msg != null && !msg.trim().isEmpty()) ? msg : null;
        })
        .filter(Objects::nonNull)
        .findFirst()
        .orElse(MessageUtils.getMessage(ErrorCode.PARAM_VALUE_NULL));

    return new Result<Void>().error(ErrorCode.PARAM_VALUE_NULL, errorMsg);
}
}

// ===== 文件: xiaozhi/common/handler/FieldMetaObjectHandler.java =====
package xiaozhi.common.handler;
```

```
import java.util.Date;

import org.apache.ibatis.reflection.MetaObject;
import org.springframework.stereotype.Component;

import com.baomidou.mybatisplus.core.handlers.MetaObjectHandler;

import xiaozhi.common.constant.Constant;
import xiaozhi.common.user.UserDetail;
import xiaozhi.modules.security.user.SecurityUser;

/**
 * 公共字段，自动填充值
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Component
public class FieldMetaObjectHandler implements MetaObjectHandler {
    private final static String CREATE_DATE = "createDate";
    private final static String CREATOR = "creator";
    private final static String UPDATE_DATE = "updateDate";
    private final static String UPDATER = "updater";

    private final static String DATA_OPERATION = "dataOperation";

    @Override
    public void insertFill(MetaObject metaObject) {
        UserDetail user = SecurityUser.getUser();
        Date date = new Date();

        // 创建者
        strictInsertFill(metaObject, CREATOR, Long.class, user.getId());
        // 创建时间 - 支持createDate和createdAt两种字段名
        if (metaObject.hasSetter(CREATE_DATE)) {
            strictInsertFill(metaObject, CREATE_DATE, Date.class, date);
        }
        if (metaObject.hasSetter("createdAt")) {
            strictInsertFill(metaObject, "createdAt", Date.class, date);
        }

        // 更新者
        strictInsertFill(metaObject, UPDATER, Long.class, user.getId());
        // 更新时间 - 支持updateDate和updatedAt两种字段名
        if (metaObject.hasSetter(UPDATE_DATE)) {
            strictInsertFill(metaObject, UPDATE_DATE, Date.class, date);
        }
        if (metaObject.hasSetter("updatedAt")) {
            strictInsertFill(metaObject, "updatedAt", Date.class, date);
        }

        // 数据标识
        strictInsertFill(metaObject, DATA_OPERATION, String.class, Constant.DataOperation.INSERT.getValue());
    }

    @Override
    public void updateFill(MetaObject metaObject) {
        Date date = new Date();

        // 更新者
```

```

        strictUpdateFill(metaObject, UPDATER, Long.class, SecurityUser.getUserId());
        // 更新时间 - 支持updateDate和updatedAt两种字段名
        if (metaObject.hasSetter(UPDATE_DATE)) {
            strictUpdateFill(metaObject, UPDATE_DATE, Date.class, date);
        }
        if (metaObject.hasSetter("updatedAt")) {
            strictUpdateFill(metaObject, "updatedAt", Date.class, date);
        }

        // 数据标识
        strictInsertFill(metaObject, DATA_OPERATION, String.class, Constant.DataOperation.UPDATE.getValue());
    }
}

// ===== 文件: xiaozhi/common/interceptor/DataFilterInterceptor.java =====
package xiaozhi.common.interceptor;

import java.util.Map;

import org.apache.ibatis.executor.Executor;
import org.apache.ibatis.mapping.BoundSql;
import org.apache.ibatis.mapping.MappedStatement;
import org.apache.ibatis.session.ResultHandler;
import org.apache.ibatis.session.RowBounds;

import com.baomidou.mybatisplus.core.toolkit.PluginUtils;
import com.baomidou.mybatisplus.extension.plugins.inner.InnerInterceptor;

import cn.hutool.core.util.StrUtil;
import net.sf.jsqlparser.JSQLParserException;
import net.sf.jsqlparser.expression.Expression;
import net.sf.jsqlparser.expression.StringValue;
import net.sf.jsqlparser.expression.operators.conditional.AndExpression;
import net.sf.jsqlparser.parser.CCJSqlParserUtil;
import net.sf.jsqlparser.statement.select.PlainSelect;
import net.sf.jsqlparser.statement.select.Select;

/**
 * 数据过滤
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
public class DataFilterInterceptor implements InnerInterceptor {

    @SuppressWarnings("rawtypes")
    @Override
    public void beforeQuery(Executor executor, MappedStatement ms, Object parameter, RowBounds rowBounds,
        ResultHandler resultHandler, BoundSql boundSql) {
        DataScope scope = getDataScope(parameter);
        // 不进行数据过滤
        if (scope == null || StrUtil.isBlank(scope.getSqlFilter())) {
            return;
        }

        // 拼接新SQL
        String buildSql = getSelect(boundSql.getSql(), scope);

        // 重写SQL
        PluginUtils.mpBoundSql(boundSql).sql(buildSql);
    }

```

```

    }

    private DataScope getDataScope(Object parameter) {
        if (parameter == null) {
            return null;
        }

        // 判断参数里是否有DataScope对象
        if (parameter instanceof Map) {
            Map<?, ?> parameterMap = (Map<?, ?>) parameter;
            for (Map.Entry<?, ?> entry : parameterMap.entrySet()) {
                if (entry.getValue() != null && entry.getValue() instanceof DataScope) {
                    return (DataScope) entry.getValue();
                }
            }
        } else if (parameter instanceof DataScope) {
            return (DataScope) parameter;
        }

        return null;
    }

    private String getSelect(String buildSql, DataScope scope) {
        try {
            Select select = (Select) CCJSqlParserUtil.parse(buildSql);
            PlainSelect plainSelect = select.getPlainSelect();

            Expression expression = plainSelect.getWhere();
            if (expression == null) {
                plainSelect.setWhere(new StringValue(scope.getSqlFilter()));
            } else {
                AndExpression andExpression = new AndExpression(expression, new StringValue(scope.getSqlFilter()));
                plainSelect.setWhere(andExpression);
            }

            return select.toString().replaceAll("'", "");
        } catch (JSQLParserException e) {
            return buildSql;
        }
    }
}

// ===== 文件: xiaozhi/common/interceptor/DataScope.java =====
package xiaozhi.common.interceptor;

/**
 * 数据范围
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
public class DataScope {
    private String sqlFilter;

    public DataScope(String sqlFilter) {
        this.sqlFilter = sqlFilter;
    }

    public String getSqlFilter() {
        return sqlFilter;
    }
}

```

```
public void setSqlFilter(String sqlFilter) {
    this.sqlFilter = sqlFilter;
}

@Override
public String toString() {
    return this.sqlFilter;
}
}

// ===== 文件: xiaozhi/common/aspect/OperationLogAspect.java =====
package xiaozhi.common.aspect;

import jakarta.servlet.http.HttpServletRequest;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import xiaozhi.common.annotation.OperationLog;
import xiaozhi.common.utils.HttpContextUtils;
import xiaozhi.common.utils.IpUtils;
import xiaozhi.modules.sys.entity.SysOperationLogEntity;
import xiaozhi.modules.sys.service.OperationLogService;

/**
 * 操作日志切面
 * <p>
 * 拦截 {@link OperationLog} 注解方法，方法调用即记录操作日志。
 * 自动填充请求路径、请求方法、IP。成功/失败按方法是否抛异常判定。
 */
@Slf4j
@Aspect
@Component
@RequiredArgsConstructor
public class OperationLogAspect {

    private final OperationLogService operationLogService;

    @Around("@annotation(operationLog)")
    public Object around(ProceedingJoinPoint point, OperationLog operationLog) throws Throwable {
        SysOperationLogEntity entity = new SysOperationLogEntity();
        entity.setOperationType(operationLog.type().name());
        entity.setOperationDesc(operationLog.type().getDesc());
        fillRequest(entity);

        try {
            Object result = point.proceed();
            entity.setStatus(1);
            operationLogService.save(entity);
            return result;
        } catch (Throwable e) {
            entity.setStatus(0);
            entity.setErrorMsg(truncate(e.getMessage()));
            operationLogService.save(entity);
            throw e;
        }
    }
}
```

```

    }

    private void fillRequest(SysOperationLogEntity entity) {
        HttpServletRequest request = HttpContextUtils.getHttpServletRequest();
        if (request != null) {
            entity.setRequestUri(request.getRequestURI());
            entity.setRequestMethod(request.getMethod());
            entity.setIp(IpUtils.getIpAddr(request));
        }
    }

    private String truncate(String msg) {
        if (msg == null) {
            return null;
        }
        return msg.length() > 500 ? msg.substring(0, 500) : msg;
    }
}

// ===== 文件: xiaozhi/common/aspect/RedisAspect.java =====
package xiaozhi.common.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import lombok.extern.slf4j.Slf4j;
import xiaozhi.common.exception.ErrorCode;
import xiaozhi.common.exception.RenException;

/**
 * Redis切面处理类
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Slf4j
@Aspect
@Component
public class RedisAspect {
    /**
     * 是否开启redis缓存 true开启 false关闭
     */
    @Value("${renren.redis.open}")
    private boolean open;

    @Around("execution(* xiaozhi.common.redis.RedisUtils.*(..))")
    public Object around(ProceedingJoinPoint point) throws Throwable {
        Object result = null;
        if (open) {
            try {
                result = point.proceed();
            } catch (Exception e) {
                log.error("redis error", e);
                throw new RenException(ErrorCode.REDIS_ERROR);
            }
        }
        return result;
    }
}

```

```
// ===== 文件: xiaozhi/common/constant/Constant.java =====
package xiaozhi.common.constant;

import lombok.Getter;

/**
 * 常量
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
public interface Constant {
    /**
     * 成功
     */
    int SUCCESS = 1;
    /**
     * 失败
     */
    int FAIL = 0;
    /**
     * OK
     */
    String OK = "OK";
    /**
     * 用户标识
     */
    String USER_KEY = "userId";
    /**
     * 菜单根节点标识
     */
    Long MENU_ROOT = 0L;
    /**
     * 部门根节点标识
     */
    Long DEPT_ROOT = 0L;
    /**
     * 数据字典根节点标识
     */
    Long DICT_ROOT = 0L;
    /**
     * 升序
     */
    String ASC = "asc";
    /**
     * 降序
     */
    String DESC = "desc";
    /**
     * 创建时间字段名
     */
    String CREATE_DATE = "create_date";

    /**
     * 创建时间字段名
     */
    String ID = "id";

    /**
     * 数据权限过滤
     */
}
```

```
String SQL_FILTER = "sqlFilter";

/**
 * 当前页码
 */
String PAGE = "page";
/**
 * 每页显示记录数
 */
String LIMIT = "limit";
/**
 * 排序字段
 */
String ORDER_FIELD = "orderField";
/**
 * 排序方式
 */
String ORDER = "order";

/**
 * 请求头授权标识
 */
String AUTHORIZATION = "Authorization";

/**
 * 服务器密钥
 */
String SERVER_SECRET = "server.secret";

/**
 * SM2公钥
 */
String SM2_PUBLIC_KEY = "server.public_key";

/**
 * SM2私钥
 */
String SM2_PRIVATE_KEY = "server.private_key";

/**
 * websocket地址
 */
String SERVER_WEBSOCKET = "server.websocket";

/**
 * mqtt gateway 配置
 */
String SERVER_MQTT_GATEWAY = "server.mqtt_gateway";

/**
 * ota地址
 */
String SERVER_OTA = "server.ota";

/**
 * 是否允许用户注册
 */
String SERVER_ALLOW_USER_REGISTER = "server.allow_user_register";

/**
```

```
* 下发六位验证码时显示的控制面板地址
*/
String SERVER_FRONTED_URL = "server.fronted_url";

/**
 * 路径分割符
 */
String FILE_EXTENSION_SEG = ".";

/**
 * mcp接入点路径
 */
String SERVER_MCP_ENDPOINT = "server.mcp_endpoint";

/**
 * mcp接入点路径
 */
String SERVER_VOICE_PRINT = "server.voice_print";

/**
 * mqtt密钥
 */
String SERVER_MQTT_SECRET = "server.mqtt_signature_key";

/**
 * WebSocket认证开关
 */
String SERVER_AUTH_ENABLED = "server.auth.enabled";

/**
 * 系统功能菜单配置
 */
String SYSTEM_WEB_MENU = "system-web.menu";

/**
 * 无记忆
 */
String MEMORY_NO_MEM = "Memory_nomem";

/**
 * 仅上报聊天记录（不总结记忆）
 */
String MEMORY_MEM_REPORT_ONLY = "Memory_mem_report_only";

/**
 * Mem0AI记忆
 */
String MEMORY_MEM0AI = "Memory_mem0ai";

/**
 * PowerMem记忆
 */
String MEMORY_POWERMEM = "Memory_powermem";

/**
 * 火山引擎双声道语音克隆
 */
String VOICE_CLONE_HUOSHAN_DOUBLE_STREAM = "huoshan_double_stream";

/**
```

```
* RAG配置类型
*/
String RAG_CONFIG_TYPE = "RAG";

enum SysBaseParam {
    /**
     * ICP备案号
     */
    BEIAN_ICP_NUM("server.beian_icp_num"),
    /**
     * GA备案号
     */
    BEIAN_GA_NUM("server.beian_ga_num"),
    /**
     * 系统名称
     */
    SERVER_NAME("server.name");

    private String value;

    SysBaseParam(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}

/**
 * 训练状态
 */
enum TrainStatus {
    /**
     * 未训练
     */
    NOT_TRAINED(0),
    /**
     * 训练中
     */
    TRAINING(1),
    /**
     * 已训练
     */
    TRAINED(2),
    /**
     * 训练失败
     */
    TRAIN_FAILED(3);

    private final int code;

    TrainStatus(int code) {
        this.code = code;
    }

    public int getCode() {
        return code;
    }
}
```

```
/**
 * 系统短信
 */
enum SysMSMParam {
    /**
     * 阿里云授权keyID
     */
    ALIYUN_SMS_ACCESS_KEY_ID("aliyun.sms.access_key_id"),
    /**
     * 阿里云授权密钥
     */
    ALIYUN_SMS_ACCESS_KEY_SECRET("aliyun.sms.access_key_secret"),
    /**
     * 阿里云短信签名
     */
    ALIYUN_SMS_SIGN_NAME("aliyun.sms.sign_name"),
    /**
     * 阿里云短信模板
     */
    ALIYUN_SMS_SMS_CODE_TEMPLATE_CODE("aliyun.sms.sms_code_template_code"),
    /**
     * 单号码最大短信发送条数
     */
    SERVER_SMS_MAX_SEND_COUNT("server.sms_max_send_count"),
    /**
     * 是否开启手机注册
     */
    SERVER_ENABLE_MOBILE_REGISTER("server.enable_mobile_register");

    private String value;

    SysMSMParam(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}

/**
 * 系统邮件
 */
enum SysEmailParam {
    /**
     * 阿里云授权keyID
     */
    ALIYUN_DM_ACCESS_KEY_ID("aliyun.access_key_id"),
    /**
     * 阿里云授权密钥
     */
    ALIYUN_DM_ACCESS_KEY_SECRET("aliyun.access_key_secret"),
    /**
     * 控制台配置的发信地址
     */
    ALIYUN_DM_ACCOUNT_NAME("aliyun.dm.account_name"),
    /**
     * 发信人昵称
     */
}
```

```

    ALIYUN_DM_FROM_ALIAS("aliyun.dm.from_alias"),
    /**
     * 邮件标签
     */
    ALIYUN_DM_TAG_NAME("aliyun.dm.tag_name"),
    /**
     * 是否启用回信地址
     */
    ALIYUN_DM_REPLY_TO_ADDRESS("aliyun.dm.reply_to_address");

    private String value;

    SysEmailParam(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}

/**
 * 数据状态
 */
enum DataOperation {
    /**
     * 插入
     */
    INSERT("I"),
    /**
     * 已修改
     */
    UPDATE("U"),
    /**
     * 已删除
     */
    DELETE("D");

    private String value;

    DataOperation(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}

@Getter
enum ChatHistoryConfEnum {
    IGNORE(0, "不记录"),
    RECORD_TEXT(1, "记录文本"),
    RECORD_TEXT_AUDIO(2, "文本音频都记录");

    private final int code;
    private final String name;

    ChatHistoryConfEnum(int code, String name) {
        this.code = code;
    }
}

```

```

        this.name = name;
    }
}

/**
 * 版本号
 */
public static final String VERSION = "1.0.0";

/**
 * 无效固件URL
 */
String INVALID_FIRMWARE_URL = "http://xiaozhi.server.com:8002/xiaozhi/otaMag/download/NOT_ACTIVATED_
FIRMWARE_THIS_IS_A_INVALID_URL";

/**
 * 字典类型
 */
enum DictType {
    /**
     * 手机区号
     */
    MOBILE_AREA("MOBILE_AREA");

    private String value;

    DictType(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}
}

// ===== 文件: xiaozhi/common/utils/AESUtils.java =====
package xiaozhi.common.utils;

import java.nio.charset.StandardCharsets;
import java.util.Base64;

import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;

public class AESUtils {

    private static final String ALGORITHM = "AES";
    private static final String TRANSFORMATION = "AES/ECB/PKCS5Padding";

    /**
     * AES加密
     *
     * @param key 密钥（16位、24位或32位）
     * @param plainText 待加密字符串
     * @return 加密后的Base64字符串
     */
    public static String encrypt(String key, String plainText) {
        try {
            // 确保密钥长度为16、24或32位
            byte[] keyBytes = padKey(key.getBytes(StandardCharsets.UTF_8));

```

```

        SecretKeySpec secretKey = new SecretKeySpec(keyBytes, ALGORITHM);

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);

        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(encryptedBytes);
    } catch (Exception e) {
        throw new RuntimeException("AES加密失败", e);
    }
}

/**
 * AES解密
 *
 * @param key          密钥（16位、24位或32位）
 * @param encryptedText 待解密的Base64字符串
 * @return 解密后的字符串
 */
public static String decrypt(String key, String encryptedText) {
    try {
        // 确保密钥长度为16、24或32位
        byte[] keyBytes = padKey(key.getBytes(StandardCharsets.UTF_8));
        SecretKeySpec secretKey = new SecretKeySpec(keyBytes, ALGORITHM);

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        cipher.init(Cipher.DECRYPT_MODE, secretKey);

        byte[] encryptedBytes = Base64.getDecoder().decode(encryptedText);
        byte[] decryptedBytes = cipher.doFinal(encryptedBytes);
        return new String(decryptedBytes, StandardCharsets.UTF_8);
    } catch (Exception e) {
        throw new RuntimeException("AES解密失败", e);
    }
}

/**
 * 填充密钥到指定长度（16、24或32位）
 *
 * @param keyBytes 原始密钥字节数组
 * @return 填充后的密钥字节数组
 */
private static byte[] padKey(byte[] keyBytes) {
    int keyLength = keyBytes.length;
    if (keyLength == 16 || keyLength == 24 || keyLength == 32) {
        return keyBytes;
    }

    // 如果密钥长度不足，用0填充；如果超过，截取前32位
    byte[] paddedKey = new byte[32];
    System.arraycopy(keyBytes, 0, paddedKey, 0, Math.min(keyLength, 32));
    return paddedKey;
}
}

// ===== 文件: xiaozhi/common/utils/ConvertUtils.java =====
package xiaozhi.common.utils;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

```

```
import org.springframework.beans.BeanUtils;

import lombok.extern.slf4j.Slf4j;

/**
 * 转换工具类
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
@Slf4j
public class ConvertUtils {
    public static <T> T sourceToTarget(Object source, Class<T> target) {
        if (source == null) {
            return null;
        }
        T targetObject = null;
        try {
            targetObject = target.getDeclaredConstructor().newInstance();
            BeanUtils.copyProperties(source, targetObject);
        } catch (Exception e) {
            log.error("convert error ", e);
        }

        return targetObject;
    }

    public static <T> List<T> sourceToTarget(Collection<?> sourceList, Class<T> target) {
        if (sourceList == null) {
            return null;
        }

        List<T> targetList = new ArrayList<>(sourceList.size());
        try {
            for (Object source : sourceList) {
                T targetObject = target.getDeclaredConstructor().newInstance();
                BeanUtils.copyProperties(source, targetObject);
                targetList.add(targetObject);
            }
        } catch (Exception e) {
            log.error("convert error ", e);
        }

        return targetList;
    }
}

// ===== 文件: xiaozhi/common/utils/DateUtils.java =====
package xiaozhi.common.utils;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.temporal.ChronoUnit;
import java.util.Date;

/**
 * 日期处理
 * Copyright (c) 人人开源 All rights reserved.
 * Website: https://www.renren.io
 */
```

```
*/
public class DateUtils {
    /**
     * 时间格式(yyyy-MM-dd)
     */
    public final static String DATE_PATTERN = "yyyy-MM-dd";
    /**
     * 时间格式(yyyy-MM-dd HH:mm:ss)
     */
    public final static String DATE_TIME_PATTERN = "yyyy-MM-dd HH:mm:ss";
    public final static String DATE_TIME_MILLIS_PATTERN = "yyyy-MM-dd HH:mm:ss.SSS";

    /**
     * 日期格式化 日期格式为: yyyy-MM-dd
     *
     * @param date 日期
     * @return 返回yyyy-MM-dd格式日期
     */
    public static String format(Date date) {
        return format(date, DATE_PATTERN);
    }

    /**
     * 日期格式化 日期格式为: yyyy-MM-dd
     *
     * @param date 日期
     * @param pattern 格式, 如: DateUtils.DATE_TIME_PATTERN
     * @return 返回yyyy-MM-dd格式日期
     */
    public static String format(Date date, String pattern) {
        if (date != null) {
            SimpleDateFormat df = new SimpleDateFormat(pattern);
            return df.format(date);
        }
        return null;
    }

    /**
     * 日期解析
     *
     * @param date 日期
     * @param pattern 格式, 如: DateUtils.DATE_TIME_PATTERN
     * @return 返回Date
     */
    public static Date parse(String date, String pattern) {
        try {
            return new SimpleDateFormat(pattern).parse(date);
        } catch (ParseException e) {
            e.printStackTrace();
        }
        return null;
    }

    public static String getDateTimeNow() {
        return getDateTimeNow(DATE_TIME_PATTERN);
    }

    public static String getDateTimeNow(String pattern) {
```

```

        return format(new Date(), pattern);
    }

    public static String millsToSecond(long mills) {
        return String.format("%.3f", mills / 1000.0);
    }

    /**
     * 获取简短的时间字符串: 10秒前返回刚刚, 多少秒前, 几小时前, 超过一周返回年月日时分秒
     * @param date
     * @return
     */
    public static String getShortTime(Date date) {
        if (date == null) {
            return null;
        }
        // 将 Date 转换为 Instant
        LocalDateTime localDateTime = date.toInstant()
            // 获取系统默认时区
            .atZone(ZoneId.systemDefault())
            // 转换为 LocalDateTime
            .toLocalDateTime();

        // 当前时间
        LocalDateTime now = LocalDateTime.now();
        // 时间差, 单位为秒
        long secondsBetween = ChronoUnit.SECONDS.between(localDateTime, now);

        if (secondsBetween <= 10) {
            return "刚刚";
        } else if (secondsBetween < 60) {
            return secondsBetween + "秒前";
        } else if (secondsBetween < 60 * 60) {
            return secondsBetween / 60 + "分钟前";
        } else if (secondsBetween < 86400) {
            return secondsBetween / 3600 + "小时前";
        } else if (secondsBetween < 604800) {
            return secondsBetween / 86400 + "天前";
        } else {
            // 超过一周, 显示完整日期时间
            return format(date, DATE_TIME_PATTERN);
        }
    }
}

// ===== 文件: xiaozhi/common/utils/HashEncryptionUtil.java =====
package xiaozhi.common.utils;

import lombok.extern.slf4j.Slf4j;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

/**
 * 哈希加密算法的工具类
 * @author zjy
 */
@Slf4j
public class HashEncryptionUtil {
    /**
     * 使用md5进行加密
     * @param context 被加密的内容

```

```
}
// ===== 文件: xiaozhi/modules/item/service/ItemFulfillmentService.java =====
package xiaozhi.modules.item.service;

import xiaozhi.modules.payment.entity.PaymentOrderEntity;

/**
 * 道具履约服务（支付回调驱动）
 */
public interface ItemFulfillmentService {

    /** 支付成功后履约：发放道具 */
    void fulfill(PaymentOrderEntity order);

    /** 退款时反向处理：扣减剩余库存（不足则记审计） */
    void rollback(PaymentOrderEntity order);
}
// ===== 文件: xiaozhi/modules/item/service/ItemService.java =====
package xiaozhi.modules.item.service;

import xiaozhi.modules.item.entity.ItemSkuEntity;
import xiaozhi.modules.item.vo.ItemSkuVO;
import xiaozhi.modules.item.vo.UserItemVO;

import java.util.List;

/**
 * 道具服务
 */
public interface ItemService {

    /** 列出 SKU; category 为 null 返回全部上架 */
    List<ItemSkuVO> listSkus(String category);

    /** 根据 SKU 编码查询；不存在返回 null */
    ItemSkuEntity getBySkuCode(String skuCode);

    /** 根据 SKU id 查询；不存在抛 ITEM_SKU_NOT_FOUND */
    ItemSkuEntity getBySkuId(Long skuId);

    /** 列出当前用户全部道具库存 */
    List<UserItemVO> myInventory(Long userId);

    /**
     * 发放道具到用户库存（幂等：基于 source_ref + sku_code 唯一索引去重）
     */
    void grant(Long userId, String skuCode, int count, String source, String sourceRef);

    /**
     * 消耗用户道具：库存不足抛 ITEM_INSUFFICIENT
     * - 服装类(outfit) 仅校验拥有（remain>0），不递减
     * - 其余按 count 扣减
     */
    void consume(Long userId, String skuCode, int count, String bizType, String bizRefId);

    /** 用户是否拥有某 outfit / sku (remain>0) */
    boolean owns(Long userId, String skuCode);
}
// ===== 文件: xiaozhi/modules/item/service/impl/ItemFulfillmentServiceImpl.java =====
package xiaozhi.modules.item.service.impl;
```

```
import cn.hutool.json.JSONObject;
import cn.hutool.json.JSONUtil;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import xiaozhi.modules.item.enums.ItemGrantSource;
import xiaozhi.modules.item.service.ItemFulfillmentService;
import xiaozhi.modules.item.service.ItemService;
import xiaozhi.modules.payment.entity.PaymentOrderEntity;

@Slf4j
@Service
@RequiredArgsConstructor
public class ItemFulfillmentServiceImpl implements ItemFulfillmentService {

    private final ItemService itemService;

    /** 道具履约：从订单快照提取skuCode并发放到用户库存 */
    @Override
    @Transactional(rollbackFor = Exception.class)
    public void fulfill(PaymentOrderEntity order) {
        // 从快照中拿 sku_code (防 sku 表后续被改)
        JSONObject snapshot = parseSnapshot(order.getProductSnapshot());
        String skuCode = snapshot.getStr("skuCode");
        if (skuCode == null || skuCode.isBlank()) {
            log.error("订单快照缺少 skuCode: orderId={}", order.getId());
            return;
        }
        int count = order.getQuantity() != null ? order.getQuantity() : 1;
        itemService.grant(order.getUserId(), skuCode, count, ItemGrantSource.PURCHASE, order.getOutTradeNo());
        log.info("道具履约完成 userId={}, sku={}, count={}, ref={}",
            order.getUserId(), skuCode, count, order.getOutTradeNo());
    }

    /** 道具退款回滚（当前不自动减库存，留运营人工处理） */
    @Override
    public void rollback(PaymentOrderEntity order) {
        // 退款回退道具：当前实现不主动减库存（避免误伤已使用部分）；
        // 留待运营在管理后台基于 item_grant_log + item_consume_log 比对人工处理。
        log.info("道具退款回滚（人工处理建议） orderId={}, outTradeNo={}", order.getId(), order.getOutTradeNo());
    }

    private static JSONObject parseSnapshot(String snapshot) {
        if (snapshot == null || snapshot.isBlank()) {
            return new JSONObject();
        }
        try {
            return JSONUtil.parseObj(snapshot);
        } catch (Exception e) {
            return new JSONObject();
        }
    }
}

// ===== 文件：xiaozhi/modules/item/service/impl/ItemServiceImpl.java =====
package xiaozhi.modules.item.service.impl;
```

```
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringUtils;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import xiaozhi.common.exception.ErrorCode;
import xiaozhi.common.exception.RenException;
import xiaozhi.modules.item.dao.ItemConsumeLogDao;
import xiaozhi.modules.item.dao.ItemGrantLogDao;
import xiaozhi.modules.item.dao.ItemSkuDao;
import xiaozhi.modules.item.dao.UserItemDao;
import xiaozhi.modules.item.entity.ItemConsumeLogEntity;
import xiaozhi.modules.item.entity.ItemGrantLogEntity;
import xiaozhi.modules.item.entity.ItemSkuEntity;
import xiaozhi.modules.item.entity.UserItemEntity;
import xiaozhi.modules.item.enums.ItemCategory;
import xiaozhi.modules.item.service.ItemService;
import xiaozhi.modules.item.vo.ItemSkuVO;
import xiaozhi.modules.item.vo.UserItemVO;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Slf4j
@Service
@RequiredArgsConstructor
public class ItemServiceImpl implements ItemService {

    private final ItemSkuDao itemSkuDao;
    private final UserItemDao userItemDao;
    private final ItemGrantLogDao grantLogDao;
    private final ItemConsumeLogDao consumeLogDao;

    /** 查询上架的道具SKU列表，可按分类过滤 */
    @Override
    public List<ItemSkuVO> listSkus(String category) {
        QueryWrapper<ItemSkuEntity> wrapper = new QueryWrapper<>();
        wrapper.eq("status", 1);
        if (StringUtils.isNotBlank(category)) {
            wrapper.eq("category", category);
        }
        wrapper.orderByAsc("sort");
        List<ItemSkuEntity> entities = itemSkuDao.selectList(wrapper);
        List<ItemSkuVO> result = new ArrayList<>(entities.size());
        for (ItemSkuEntity entity : entities) {
            result.add(ItemSkuVO.toVO(entity));
        }
        return result;
    }

    /** 根据道具编码查询SKU实体 */
    @Override
    public ItemSkuEntity getBySkuCode(String skuCode) {
        if (StringUtils.isBlank(skuCode)) {
            return null;
        }
    }
```

```
        return itemSkuDao.selectOne(new QueryWrapper<ItemSkuEntity>().eq("sku_code", skuCode));
    }

    /** 根据道具ID查询SKU实体, 不存在则抛异常 */
    @Override
    public ItemSkuEntity getBySkuId(Long skuId) {
        if (skuId == null) {
            throw new RenException(ErrorCode.ITEM_SKU_NOT_FOUND);
        }
        ItemSkuEntity entity = itemSkuDao.selectById(skuId);
        if (entity == null) {
            throw new RenException(ErrorCode.ITEM_SKU_NOT_FOUND);
        }
        return entity;
    }

    /** 查询用户道具库存 */
    @Override
    public List<UserItemV0> myInventory(Long userId) {
        QueryWrapper<UserItemEntity> wrapper = new QueryWrapper<>();
        wrapper.eq("user_id", userId);
        List<UserItemEntity> items = userItemDao.selectList(wrapper);
        if (items.isEmpty()) {
            return List.of();
        }

        // 批量查询 SKU 元信息
        List<String> codes = items.stream().map(UserItemEntity::getSkuCode).toList();
        QueryWrapper<ItemSkuEntity> skuWrapper = new QueryWrapper<>();
        skuWrapper.in("sku_code", codes);
        List<ItemSkuEntity> skus = itemSkuDao.selectList(skuWrapper);
        Map<String, ItemSkuEntity> skuMap = new HashMap<>();
        for (ItemSkuEntity sku : skus) {
            skuMap.put(sku.getSkuCode(), sku);
        }

        List<UserItemV0> result = new ArrayList<>(items.size());
        for (UserItemEntity item : items) {
            ItemSkuEntity sku = skuMap.get(item.getSkuCode());
            String name = sku != null ? sku.getSkuName() : item.getSkuCode();
            String cat = sku != null ? sku.getCategory() : "";
            result.add(UserItemV0.toV0(item, name, cat));
        }
        return result;
    }

    /** 发放道具到用户库存（幂等，基于sourceRef+skuCode去重） */
    @Override
    @Transactional(rollbackFor = Exception.class)
    public void grant(Long userId, String skuCode, int count, String source, String sourceRef) {
        if (count <= 0) {
            return;
        }
        ItemSkuEntity sku = getBySkuCode(skuCode);
        if (sku == null) {
            throw new RenException(ErrorCode.ITEM_SKU_NOT_FOUND);
        }

        // 1) 写入发放流水（基于 uk_source_ref_sku 实现幂等）
        ItemGrantLogEntity grant = new ItemGrantLogEntity();
```

```

grant.setUserId(userId);
grant.setSkuCode(skuCode);
grant.setCount(count);
grant.setSource(source);
grant.setSourceRef(sourceRef);
try {
    grantLogDao.insert(grant);
} catch (DuplicateKeyException dup) {
    // 已发放过，幂等返回
    log.info("道具已发放过，跳过: userId={}, sku={}, ref={}", userId, skuCode, sourceRef);
    return;
}

// 2) 更新或创建用户库存（处理并发 INSERT 冲突）
UserItemEntity userItem = userItemDao.selectOne(
    new QueryWrapper<UserItemEntity>().eq("user_id", userId).eq("sku_code", skuCode));
if (userItem == null) {
    try {
        userItem = new UserItemEntity();
        userItem.setUserId(userId);
        userItem.setSkuCode(skuCode);
        userItem.setTotalCount(count);
        userItem.setUsedCount(0);
        userItem.setRemainCount(count);
        userItemDao.insert(userItem);
        // INSERT 成功，库存已就绪
        return;
    } catch (DuplicateKeyException e) {
        // 并发场景：另一事务已插入，重查后走更新
        log.info("用户库存并发插入冲突，回退到更新: userId={}, sku={}", userId, skuCode);
        userItem = userItemDao.selectOne(
            new QueryWrapper<UserItemEntity>().eq("user_id", userId).eq("sku_code", skuCode)
        );
    }
}
// 已有记录（初始查到 或 并发重查），累加库存
if (userItem == null) {
    throw new IllegalStateException("用户库存记录不应为空: userId=" + userId + ", sku=" + skuCod
e);
}
userItem.setTotalCount(userItem.getTotalCount() + count);
userItem.setRemainCount(userItem.getRemainCount() + count);
userItemDao.updateById(userItem);
}

/** 消耗道具（服装类仅校验拥有不扣减，其余按次原子扣减） */
@Override
@Transactional(rollbackFor = Exception.class)
public void consume(Long userId, String skuCode, int count, String bizType, String bizRefId) {
    if (count <= 0) {
        return;
    }
    ItemSkuEntity sku = getBySkuCode(skuCode);
    if (sku == null) {
        throw new RenException(ErrorCodes.ITEM_SKU_NOT_FOUND);
    }

    if (ItemCategory.isOutfit(sku.getCategory())) {
        // 服装：拥有即可，不递减
        if (!owns(userId, skuCode)) {

```

```

        throw new RenException(ErrorCode.ITEM_INSUFFICIENT);
    }
    // 记录穿戴流水
    recordConsume(userId, skuCode, 0, bizType, bizRefId, "outfit equip");
    return;
}

// 一次性消耗: 原子扣减
int affected = userItemDao.deductRemain(userId, skuCode, count);
if (affected <= 0) {
    throw new RenException(ErrorCode.ITEM_INSUFFICIENT);
}
recordConsume(userId, skuCode, count, bizType, bizRefId, null);
}

/** 判断用户是否拥有指定道具 */
@Override
public boolean owns(Long userId, String skuCode) {
    UserItemEntity ui = userItemDao.selectOne(
        new QueryWrapper<UserItemEntity>().eq("user_id", userId).eq("sku_code", skuCode));
    return ui != null && ui.getRemainCount() != null && ui.getRemainCount() > 0;
}

private void recordConsume(Long userId, String skuCode, int count, String bizType, String bizRefId,
String remark) {
    ItemConsumeLogEntity entry = new ItemConsumeLogEntity();
    entry.setUserId(userId);
    entry.setSkuCode(skuCode);
    entry.setCount(count);
    entry.setBizType(bizType);
    entry.setBizRefId(bizRefId);
    entry.setRemark(remark);
    consumeLogDao.insert(entry);
}
}

// ===== 文件: xiaozhi/modules/correctword/dto/CorrectWordFileCreatedTO.java =====
package xiaozhi.modules.correctword.dto;

import java.util.List;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import lombok.Data;

@Data
@Schema(description = "创建替换词文件DTO")
public class CorrectWordFileCreatedTO {

    @NotBlank(message = "文件名不能为空")
    @Schema(description = "文件名")
    private String fileName;

    @NotEmpty(message = "替换词内容不能为空")
    @Schema(description = "替换词内容, 每条格式: 原词|替换词")
    private List<String> content;

    @Schema(description = "文件大小(字节), 不能超过1MB")
    private Long fileSize;
}

```

```
// ===== 文件: xiaozhi/modules/correctword/vo/CorrectWordFileV0.java =====
package xiaozhi.modules.correctword.vo;

import java.util.Date;
import java.util.List;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

@Data
@Schema(description = "替换词文件列表V0")
public class CorrectWordFileV0 {

    @Schema(description = "替换词文件ID")
    private String id;

    @Schema(description = "原始文件名")
    private String fileName;

    @Schema(description = "替换词数量")
    private Integer wordCount;

    @Schema(description = "替换词内容，每行一条")
    private List<String> content;

    @Schema(description = "创建时间")
    private Date createdAt;

    @Schema(description = "更新时间")
    private Date updatedAt;
}

// ===== 文件: xiaozhi/modules/correctword/vo/CorrectWordSimpleV0.java =====
package xiaozhi.modules.correctword.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

@Data
@Schema(description = "替换词精简V0（设备端使用）")
public class CorrectWordSimpleV0 {

    @Schema(description = "原词")
    private String sourceWord;

    @Schema(description = "替换词")
    private String targetWord;
}

// ===== 文件: xiaozhi/modules/correctword/entity/CorrectWordFileEntity.java =====
package xiaozhi.modules.correctword.entity;

import java.util.Date;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

@Data
```

```
@TableName("ai_agent_correct_word_file")
@Schema(description = "智能体替换词文件")
public class CorrectWordFileEntity {

    @TableId(type = IdType.ASSIGN_UUID)
    @Schema(description = "替换词文件ID")
    private String id;

    @Schema(description = "原始文件名")
    private String fileName;

    @Schema(description = "替换词数量")
    private Integer wordCount;

    @Schema(description = "文件原始内容（用于下载）")
    private String content;

    @Schema(description = "创建者")
    private Long creator;

    @Schema(description = "创建时间")
    private Date createdAt;

    @Schema(description = "更新者")
    private Long updater;

    @Schema(description = "更新时间")
    private Date updatedAt;
}

// ===== 文件: xiaozhi/modules/correctword/entity/CorrectWordItemEntity.java =====
package xiaozhi.modules.correctword.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

@Data
@TableName("ai_agent_correct_word_item")
@Schema(description = "替换词词条")
public class CorrectWordItemEntity {

    @TableId(type = IdType.ASSIGN_UUID)
    @Schema(description = "词条ID")
    private String id;

    @Schema(description = "所属文件ID")
    private String fileId;

    @Schema(description = "原词")
    private String sourceWord;

    @Schema(description = "替换词")
    private String targetWord;
}

// ===== 文件: xiaozhi/modules/correctword/dao/CorrectWordFileDao.java =====
package xiaozhi.modules.correctword.dao;
```

```
import org.apache.ibatis.annotations.Mapper;

import xiaozhi.common.dao.BaseDao;
import xiaozhi.modules.correctword.entityCorrectWordFileEntity;

@Mapper
public interface CorrectWordFileDao extends BaseDao<CorrectWordFileEntity> {
}

// ===== 文件: xiaozhi/modules/correctword/dao/CorrectWordItemDao.java =====
package xiaozhi.modules.correctword.dao;

import java.util.List;

import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;

import xiaozhi.common.dao.BaseDao;
import xiaozhi.modules.correctword.entityCorrectWordItemEntity;

@Mapper
public interface CorrectWordItemDao extends BaseDao<CorrectWordItemEntity> {

    int batchInsert(@Param("list") List<CorrectWordItemEntity> items);
}

// ===== 文件: xiaozhi/modules/correctword/controller/CorrectWordController.java =====
package xiaozhi.modules.correctword.controller;

import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;

import org.apache.shiro.authz.annotation.RequiresPermissions;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.Parameters;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.AllArgsConstructor;
import xiaozhi.common.constant.Constant;
import xiaozhi.common.page.PageData;
import xiaozhi.common.utils.Result;
import xiaozhi.modules.correctword.dtoCorrectWordFileCreatedTO;
import xiaozhi.modules.correctword.serviceCorrectWordFileService;
import xiaozhi.modules.correctword.voCorrectWordFileVO;

@RestController
```

```

@RequestMapping("/correct-word")
@Tag(name = "替换词管理")
@AllArgsConstructor
public class CorrectWordController {

    private final CorrectWordFileService correctWordFileService;

    @PostMapping("/file")
    @Operation(summary = "创建替换词文件")
    @RequiresPermissions("sys:role:normal")
    public Result<CorrectWordFileVO> createFile(@Valid @RequestBody CorrectWordFileCreatedDTO dto) {
        CorrectWordFileVO vo = correctWordFileService.createFile(dto);
        return new Result<CorrectWordFileVO>().ok(vo);
    }

    @PutMapping("/file/{fileId}")
    @Operation(summary = "修改替换词文件")
    @RequiresPermissions("sys:role:normal")
    public Result<Void> updateFile(@PathVariable String fileId, @Valid @RequestBody CorrectWordFileCreatedDTO dto) {
        correctWordFileService.updateFile(fileId, dto);
        return new Result<>();
    }

    @GetMapping("/file/list")
    @Operation(summary = "分页获取当前用户替换词文件列表")
    @RequiresPermissions("sys:role:normal")
    @Parameters({
        @Parameter(name = Constant.PAGE, description = "当前页码, 从1开始", required = true),
        @Parameter(name = Constant.LIMIT, description = "每页显示记录数", required = true),
    })
    public Result<PageData<CorrectWordFileVO>> listFiles(
        @Parameter(hidden = true) @RequestParam Map<String, Object> params) {
        PageData<CorrectWordFileVO> page = correctWordFileService.listFiles(params);
        return new Result<PageData<CorrectWordFileVO>>().ok(page);
    }

    @GetMapping("/file/select")
    @Operation(summary = "智能体获取当前用户替换词文件列表")
    @RequiresPermissions("sys:role:normal")
    public Result<List<CorrectWordFileVO>> listAllFiles() {
        List<CorrectWordFileVO> list = correctWordFileService.listAllFiles();
        return new Result<List<CorrectWordFileVO>>().ok(list);
    }

    @GetMapping("/file/download/{fileId}")
    @Operation(summary = "下载替换词文件")
    @RequiresPermissions("sys:role:normal")
    public ResponseEntity<byte[]> downloadFile(@PathVariable String fileId) {
        CorrectWordFileVO vo = correctWordFileService.getFileContent(fileId);
        if (vo == null || vo.getContent() == null || vo.getContent().isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        byte[] bytes = String.join("\n", vo.getContent()).getBytes(StandardCharsets.UTF_8);
        String encodedFileName = URLEncoder.encode(vo.getFileName(), StandardCharsets.UTF_8).replace("+", "%20");
        String asciiFileName = vo.getFileName().replaceAll("[^\\x00-\\x7F]", "_");
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" + asciiFileName + "\"; filename*=UTF-8'" + encodedFil

```

```

eName)
        .contentType(MediaType.APPLICATION_OCTET_STREAM)
        .body(bytes);
    }

    @DeleteMapping("/file/{fileId}")
    @Operation(summary = "删除替换词文件")
    @RequiresPermissions("sys:role:normal")
    public Result<Void> deleteFile(@PathVariable String fileId) {
        correctWordFileService.deleteFile(fileId);
        return new Result<>();
    }

    @PostMapping("/file/batch-delete")
    @Operation(summary = "批量删除替换词文件")
    @RequiresPermissions("sys:role:normal")
    public Result<Void> batchDeleteFiles(@RequestBody List<String> fileIds) {
        if (fileIds == null || fileIds.isEmpty()) {
            return new Result<>();
        }
        correctWordFileService.batchDeleteFiles(fileIds);
        return new Result<>();
    }
}

// ===== 文件: xiaozhi/modules/correctword/service/CorrectWordFileService.java =====
package xiaozhi.modules.correctword.service;

import java.util.List;
import java.util.Map;

import xiaozhi.common.page.PageData;
import xiaozhi.modules.correctword.dto.CorrectWordFileCreatedTO;
import xiaozhi.modules.correctword.vo.CorrectWordFileVO;
import xiaozhi.modules.correctword.vo.CorrectWordSimpleVO;

public interface CorrectWordFileService {

    /**
     * 创建替换词文件
     *
     * @param dto 创建参数
     * @return 文件VO
     */
    CorrectWordFileVO createFile(CorrectWordFileCreatedTO dto);

    /**
     * 修改替换词文件（全量替换词条）
     *
     * @param fileId 文件ID
     * @param dto 修改参数
     */
    void updateFile(String fileId, CorrectWordFileCreatedTO dto);

    /**
     * 获取当前用户的替换词文件列表
     *
     * @param params 分页参数
     * @return 分页数据
     */
    PageData<CorrectWordFileVO> listFiles(Map<String, Object> params);

```

```
/**
 * 获取当前用户的替换词文件列表（不分页，用于下拉选择）
 *
 * @return 文件列表
 */
List<CorrectWordFileV0> listAllFiles();

/**
 * 获取文件原始内容（用于下载）
 *
 * @param fileId 文件ID
 * @return 文件实体
 */
CorrectWordFileV0 getFileContent(String fileId);

/**
 * 删除替换词文件及其所有词条和关联记录
 *
 * @param fileId 文件ID
 */
void deleteFile(String fileId);

/**
 * 删除智能体关联的替换词文件关联记录（不删文件本身）
 *
 * @param agentId 智能体ID
 */
void deleteMappingsByAgentId(String agentId);

/**
 * 获取智能体的所有替换词条（精简版，供设备端使用）
 *
 * @param agentId 智能体ID
 * @return 替换词列表
 */
List<CorrectWordSimpleV0> getAllItemsByAgentId(String agentId);

/**
 * 获取智能体关联的替换词文件ID列表
 *
 * @param agentId 智能体ID
 * @return 文件ID列表
 */
List<String> getAgentCorrectWordFileIds(String agentId);

/**
 * 保存智能体关联的替换词文件（全量替换）
 *
 * @param agentId 智能体ID
 * @param fileIds 文件ID列表
 */
void saveAgentCorrectWords(String agentId, List<String> fileIds);

/**
 * 批量删除替换词文件
 *
 * @param fileIds 文件ID列表
 */
void batchDeleteFiles(List<String> fileIds);
```

```
}
// ===== 文件: xiaozhi/modules/correctword/service/impl/CorrectWordFileServiceImpl.java =====
package xiaozhi.modules.correctword.service.impl;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Date;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;

import lombok.AllArgsConstructor;
import xiaozhi.common.exception.ErrorCode;
import xiaozhi.common.exception.RenException;
import xiaozhi.common.page.PageData;
import xiaozhi.common.service.impl.BaseServiceImpl;
import xiaozhi.common.utils.ConvertUtils;
import xiaozhi.modules.agent.dao.AgentCorrectWordMappingDao;
import xiaozhi.modules.correctword.dao.CorrectWordFileDao;
import xiaozhi.modules.correctword.dao.CorrectWordItemDao;
import xiaozhi.modules.correctword.dto.CorrectWordFileCreateDTO;
import xiaozhi.modules.agent.entity.AgentCorrectWordMappingEntity;
import xiaozhi.modules.correctword.entity.CorrectWordFileEntity;
import xiaozhi.modules.correctword.entity.CorrectWordItemEntity;
import xiaozhi.modules.correctword.service.CorrectWordFileService;
import xiaozhi.modules.correctword.vo.CorrectWordFileVO;
import xiaozhi.modules.correctword.vo.CorrectWordSimpleVO;
import xiaozhi.modules.security.user.SecurityUser;

@Service
@AllArgsConstructor
public class CorrectWordFileServiceImpl extends BaseServiceImpl<CorrectWordFileDao, CorrectWordFileEntity>
    implements CorrectWordFileService {

    private final CorrectWordFileDao correctWordFileDao;
    private final CorrectWordItemDao correctWordItemDao;
    private final AgentCorrectWordMappingDao agentCorrectWordMappingDao;

    @Override
    @Transactional(rollbackFor = Exception.class)
    public CorrectWordFileVO createFile(CorrectWordFileCreateDTO dto) {
        // 校验文件大小不能超过1MB
        if (dto.getFileSize() != null && dto.getFileSize() > 1024 * 1024) {
            throw new RenException(ErrorCode.FILE_SIZE_OVER_LIMIT);
        }

        // 校验文件名是否重复
        Long userId = SecurityUser.getUserId();
        LambdaQueryWrapper<CorrectWordFileEntity> nameWrapper = new LambdaQueryWrapper<>();
        nameWrapper.eq(CorrectWordFileEntity::getCreator, userId)
            .eq(CorrectWordFileEntity::getFileName, dto.getFileName());
        if (correctWordFileDao.selectCount(nameWrapper) > 0) {
            throw new RenException(ErrorCode.CORRECT_WORD_FILE_NAME_EXISTS);
        }
    }
}
```

```
}

List<CorrectWordItemEntity> items = parseContent(dto.getContent());

// 保存文件记录
CorrectWordFileEntity fileEntity = new CorrectWordFileEntity();
fileEntity.setFileName(dto.getFileName());
fileEntity.setWordCount(items.size());
fileEntity.setContent(String.join("\n", dto.getContent()));
fileEntity.setCreator(SecurityUser.getUserId());
fileEntity.setCreatedAt(new Date());
correctWordFileDao.insert(fileEntity);

// 设置fileId并批量保存词条
String fileId = fileEntity.getId();
for (CorrectWordItemEntity item : items) {
    item.setFileId(fileId);
}
if (!items.isEmpty()) {
    correctWordItemDao.batchInsert(items);
}

return toVO(fileEntity);
}

@Override
@Transactional(rollbackFor = Exception.class)
public void updateFile(String fileId, CorrectWordFileCreatedDTO dto) {
    CorrectWordFileEntity fileEntity = correctWordFileDao.selectById(fileId);
    if (fileEntity == null) {
        return;
    }

    // 校验文件名是否重复（排除自身）
    Long userId = SecurityUser.getUserId();
    LambdaQueryWrapper<CorrectWordFileEntity> nameWrapper = new LambdaQueryWrapper<>();
    nameWrapper.eq(CorrectWordFileEntity::getCreator, userId)
        .eq(CorrectWordFileEntity::getFileName, dto.getFileName())
        .ne(CorrectWordFileEntity::getId, fileId);
    if (correctWordFileDao.selectCount(nameWrapper) > 0) {
        throw new RenException("文件名已存在: " + dto.getFileName());
    }

    // 先删除旧词条
    LambdaQueryWrapper<CorrectWordItemEntity> deleteWrapper = new LambdaQueryWrapper<>();
    deleteWrapper.eq(CorrectWordItemEntity::getFileId, fileId);
    correctWordItemDao.delete(deleteWrapper);

    // 解析新词条并批量保存
    List<CorrectWordItemEntity> items = parseContent(dto.getContent());
    if (!items.isEmpty()) {
        for (CorrectWordItemEntity item : items) {
            item.setFileId(fileId);
        }
        correctWordItemDao.batchInsert(items);
    }

    // 更新文件记录
    fileEntity.setFileName(dto.getFileName());
    fileEntity.setWordCount(items.size());
}
```

```
        fileEntity.setContent(String.join("\n", dto.getContent()));
        fileEntity.setUpdater(SecurityUser.getUserId());
        fileEntity.setUpdatedAt(new Date());
        correctWordFileDao.updateById(fileEntity);
    }

    @Override
    public PageData<CorrectWordFileV0> listFiles(Map<String, Object> params) {
        Long userId = SecurityUser.getUserId();
        IPage<CorrectWordFileEntity> page = getPage(params, "created_at", false);
        LambdaQueryWrapper<CorrectWordFileEntity> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(CorrectWordFileEntity::getCreator, userId)
            .orderByDesc(CorrectWordFileEntity::getCreatedAt);
        correctWordFileDao.selectPage(page, wrapper);
        List<CorrectWordFileV0> voList = toV0List(page.getRecords());
        return new PageData<>(voList, page.getTotal());
    }

    @Override
    public List<CorrectWordFileV0> listAllFiles() {
        Long userId = SecurityUser.getUserId();
        LambdaQueryWrapper<CorrectWordFileEntity> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(CorrectWordFileEntity::getCreator, userId)
            .orderByDesc(CorrectWordFileEntity::getCreatedAt);
        List<CorrectWordFileEntity> entities = correctWordFileDao.selectList(wrapper);
        return toV0List(entities);
    }

    @Override
    public CorrectWordFileV0 getFileContent(String fileId) {
        CorrectWordFileEntity entity = correctWordFileDao.selectById(fileId);
        return toV0(entity);
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public void deleteFile(String fileId) {
        if (fileId == null || fileId.trim().isEmpty()) {
            return;
        }
        // 先删除关联表记录
        agentCorrectWordMappingDao.deleteByFileId(fileId);
        // 删除词条
        LambdaQueryWrapper<CorrectWordItemEntity> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(CorrectWordItemEntity::getFileId, fileId);
        correctWordItemDao.delete(wrapper);
        // 删除文件
        correctWordFileDao.deleteById(fileId);
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public void deleteMappingsByAgentId(String agentId) {
        agentCorrectWordMappingDao.deleteByAgentId(agentId);
    }

    @Override
    public List<CorrectWordSimpleV0> getAllItemsByAgentId(String agentId) {
        // 通过关联表获取文件ID列表
        List<AgentCorrectWordMappingEntity> mappings = agentCorrectWordMappingDao.selectByAgentId(agentId);
    }
}
```

```
d);
    if (mappings == null || mappings.isEmpty()) {
        return new ArrayList<>();
    }
    List<String> fileIds = mappings.stream()
        .map(AgentCorrectWordMappingEntity::getFileId)
        .collect(Collectors.toList());

    // 根据文件ID列表查询词条
    LambdaQueryWrapper<CorrectWordItemEntity> wrapper = new LambdaQueryWrapper<>();
    wrapper.in(CorrectWordItemEntity::getFileId, fileIds);
    List<CorrectWordItemEntity> entities = correctWordItemDao.selectList(wrapper);
    return ConvertUtils.sourceToTarget(entities, CorrectWordSimpleVO.class);
}

@Override
public List<String> getAgentCorrectWordFileIds(String agentId) {
    List<AgentCorrectWordMappingEntity> mappings = agentCorrectWordMappingDao.selectByAgentId(agentId);
    if (mappings == null || mappings.isEmpty()) {
        return new ArrayList<>();
    }
    return mappings.stream()
        .map(AgentCorrectWordMappingEntity::getFileId)
        .collect(Collectors.toList());
}

@Override
@Transactional(rollbackFor = Exception.class)
public void saveAgentCorrectWords(String agentId, List<String> fileIds) {
    // 先删除旧的关联记录
    agentCorrectWordMappingDao.deleteByAgentId(agentId);

    if (fileIds == null || fileIds.isEmpty()) {
        return;
    }

    // 批量插入新的关联记录
    Long userId = SecurityUser.getUserId();
    Date now = new Date();
    List<AgentCorrectWordMappingEntity> mappings = new ArrayList<>();
    for (String fileId : fileIds) {
        AgentCorrectWordMappingEntity mapping = new AgentCorrectWordMappingEntity();
        mapping.setAgentId(agentId);
        mapping.setFileId(fileId);
        mapping.setCreator(userId);
        mapping.setCreatedAt(now);
        mapping.setUpdater(userId);
        mapping.setUpdatedAt(now);
        mappings.add(mapping);
    }
    agentCorrectWordMappingDao.batchInsertMapping(mappings);
}

@Override
@Transactional(rollbackFor = Exception.class)
public void batchDeleteFiles(List<String> fileIds) {
    if (fileIds == null || fileIds.isEmpty()) {
        return;
    }
}
```

```
        for (String fileId : fileIds) {
            if (fileId == null || fileId.trim().isEmpty()) {
                continue;
            }
            deleteFile(fileId.trim());
        }
    }

    /**
     * 解析替换词内容，每条格式：原词|替换词
     */
    private List<CorrectWordItemEntity> parseContent(List<String> lines) {
        List<CorrectWordItemEntity> items = new ArrayList<>();
        if (lines == null || lines.isEmpty()) {
            return items;
        }
        for (String line : lines) {
            line = line.trim();
            if (line.isEmpty()) {
                continue;
            }
            int idx = line.indexOf('|');
            if (idx <= 0 || idx >= line.length() - 1) {
                continue;
            }
            String sourceWord = line.substring(0, idx).trim();
            String targetWord = line.substring(idx + 1).trim();
            if (sourceWord.isEmpty() || targetWord.isEmpty()) {
                continue;
            }
            CorrectWordItemEntity item = new CorrectWordItemEntity();
            item.setSourceWord(sourceWord);
            item.setTargetWord(targetWord);
            items.add(item);
        }
        return items;
    }

    private CorrectWordFileV0 toV0(CorrectWordFileEntity entity) {
        if (entity == null) {
            return null;
        }
        CorrectWordFileV0 vo = new CorrectWordFileV0();
        vo.setId(entity.getId());
        vo.setFileName(entity.getFileName());
        vo.setWordCount(entity.getWordCount());
        vo.setContent(entity.getContent() != null
            ? Arrays.asList(entity.getContent().split("\n"))
            : new ArrayList<>());
        vo.setCreatedAt(entity.getCreatedAt());
        vo.setUpdatedAt(entity.getUpdatedAt());
        return vo;
    }

    private List<CorrectWordFileV0> toV0List(List<CorrectWordFileEntity> entities) {
        if (entities == null || entities.isEmpty()) {
            return new ArrayList<>();
        }
        return entities.stream().map(this::toV0).collect(Collectors.toList());
    }
}
```

```
}
// ===== 文件: xiaozhi/modules/email/dto/EmailSendDTO.java =====
package xiaozhi.modules.email.dto;

import java.io.InputStream;

import lombok.Data;

/**
 * 邮件发送请求 DTO
 */
@Data
public class EmailSendDTO {

    /**
     * 收件人邮箱地址
     */
    private String toAddress;

    /**
     * 邮件主题
     */
    private String subject;

    /**
     * 邮件 HTML 正文
     */
    private String htmlBody;

    /**
     * 附件名称（可为空，为空则不携带附件）
     */
    private String attachmentName;

    /**
     * 附件内容输入流（与 attachmentName 配合使用）
     */
    private InputStream attachmentStream;
}
// ===== 文件: xiaozhi/modules/email/service/EmailService.java =====
package xiaozhi.modules.email.service;

import xiaozhi.modules.email.dto.EmailSendDTO;

/**
 * 邮件发送服务
 */
public interface EmailService {

    /**
     * 发送邮件
     *
     * @param dto 邮件发送请求
     */
    void sendEmail(EmailSendDTO dto);
}
// ===== 文件: xiaozhi/modules/email/service/impl/AliyunDirectMailService.java =====
package xiaozhi.modules.email.service.impl;

import com.aliyun.dm20151123.Client;
```

```
import com.aliyun.dm20151123.models.SingleSendMailAdvanceRequest;
import com.aliyun.dm20151123.models.SingleSendMailRequest;
import com.aliyun.dm20151123.models.SingleSendMailResponse;
import com.aliyun.teaopenapi.models.Config;
import com.aliyun.teautil.models.RuntimeOptions;

import lombok.AllArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringUtils;
import org.springframework.stereotype.Service;
import xiaozhi.common.constant.Constant;
import xiaozhi.common.exception.ErrorCode;
import xiaozhi.common.exception.RenException;
import xiaozhi.modules.email.dto.EmailSendDTO;
import xiaozhi.modules.email.service.EmailService;
import xiaozhi.modules.sys.service.SysParamsService;

/**
 * 阿里云邮件推送 (DirectMail) 实现
 */
@Slf4j
@Service
@AllArgsConstructor
public class AliyunDirectMailService implements EmailService {

    private final SysParamsService sysParamsService;

    @Override
    public void sendEmail(EmailSendDTO dto) {
        Client client = createClient();
        try {
            RuntimeOptions runtime = new RuntimeOptions();
            SingleSendMailResponse response;
            if (StringUtils.isNotBlank(dto.getAttachmentName()) && dto.getAttachmentStream() != null) {
                response = sendWithAttachment(client, dto, runtime);
            } else {
                response = sendPlain(client, dto, runtime);
            }
            log.info("邮件发送成功, requestId={}, to={}", response.getBody().getRequestId(), dto.getToAddress());
        } catch (Exception e) {
            log.error("邮件发送失败, to={}", dto.getToAddress(), e);
            throw new RenException(ErrorCode.EMAIL_SEND_FAILED);
        }
    }

    private SingleSendMailResponse sendPlain(Client client, EmailSendDTO dto, RuntimeOptions runtime) throws Exception {
        SingleSendMailRequest request = new SingleSendMailRequest()
            .setAccountName(getParam(Constant.SysEmailParam.ALIYUN_DM_ACCOUNT_NAME))
            .setAddressType(1)
            .setReplyToAddress(Boolean.parseBoolean(getParam(Constant.SysEmailParam.ALIYUN_DM_REPLY_TO_ADDRESS)))
            .setToAddress(dto.getToAddress())
            .setSubject(dto.getSubject())
            .setHtmlBody(dto.getHtmlBody())
            .setFromAlias(getParam(Constant.SysEmailParam.ALIYUN_DM_FROM_ALIAS))
            .setTagName(getParam(Constant.SysEmailParam.ALIYUN_DM_TAG_NAME));
        return client.singleSendMailWithOptions(request, runtime);
    }
}
```

```

private SingleSendMailResponse sendWithAttachment(Client client, EmailSendDTO dto, RuntimeOptions runtime)
    throws Exception {
    SingleSendMailAdvanceRequest.SingleSendMailAdvanceRequestAttachments attachment = new SingleSendMailAdvanceRequest.SingleSendMailAdvanceRequestAttachments();
    attachment.setAttachmentName(dto.getAttachmentName());
    attachment.setAttachmentUrlObject(dto.getAttachmentStream());

    SingleSendMailAdvanceRequest request = new SingleSendMailAdvanceRequest()
        .setAccountName(getParam(Constant.SysEmailParam.ALIYUN_DM_ACCOUNT_NAME))
        .setAddressType(1)
        .setReplyToAddress(Boolean.parseBoolean(getParam(Constant.SysEmailParam.ALIYUN_DM_REPLY_TO_ADDRESS)))
        .setToAddress(dto.getToAddress())
        .setSubject(dto.getSubject())
        .setHtmlBody(dto.getHtmlBody())
        .setFromAlias(getParam(Constant.SysEmailParam.ALIYUN_DM_FROM_ALIAS))
        .setTagName(getParam(Constant.SysEmailParam.ALIYUN_DM_TAG_NAME))
        .setAttachments(java.util.List.of(attachment));
    return client.singleSendMailAdvance(request, runtime);
}

/**
 * 创建阿里云 DM 连接
 */
private Client createClient() {
    String accessKeyId = getParam(Constant.SysEmailParam.ALIYUN_DM_ACCESS_KEY_ID);
    String accessKeySecret = getParam(Constant.SysEmailParam.ALIYUN_DM_ACCESS_KEY_SECRET);
    try {
        Config config = new Config()
            .setAccessKeyId(accessKeyId)
            .setAccessKeySecret(accessKeySecret);
        config.endpoint = "dm.aliyuncs.com";
        config.regionId = "cn-hangzhou";
        return new Client(config);
    } catch (Exception e) {
        log.error("建立邮件连接失败", e);
        throw new RenException(ErrorCode.EMAIL_CONNECTION_FAILED);
    }
}

private String getParam(Constant.SysEmailParam param) {
    return sysParamsService.getValue(param.getValue(), true);
}
}

// ===== 文件: xiaozhi/modules/sms/service/SmsService.java =====
package xiaozhi.modules.sms.service;

/**
 * 短信服务的方法定义接口
 *
 * @author zjy
 * @since 2025-05-12
 */
public interface SmsService {

    /**
     * 发送验证码短信
     * @param phone 手机号码

```

```

    * @param VerificationCode 验证码
    */
    void sendVerificationCodeSms(String phone, String VerificationCode) ;
}
// ===== 文件: xiaozhi/modules/sms/service/imp/ALiYunSmsService.java =====
package xiaozhi.modules.sms.service.imp;

import com.aliyun.dysmsapi20170525.Client;
import com.aliyun.dysmsapi20170525.models.SendSmsRequest;
import com.aliyun.dysmsapi20170525.models.SendSmsResponse;
import com.aliyun.teaopenapi.models.Config;
import com.aliyun.teautil.models.RuntimeOptions;
import lombok.AllArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import xiaozhi.common.constant.Constant;
import xiaozhi.common.exception.ErrorCode;
import xiaozhi.common.exception.RenException;
import xiaozhi.common.redis.RedisKeys;
import xiaozhi.common.redis.RedisUtils;
import xiaozhi.modules.sms.service.SmsService;
import xiaozhi.modules.sys.service.SysParamsService;

@Service
@AllArgsConstructor
@Slf4j
public class ALiYunSmsService implements SmsService {
    private final SysParamsService sysParamsService;
    private final RedisUtils redisUtils;

    @Override
    public void sendVerificationCodeSms(String phone, String VerificationCode) {
        Client client = createClient();
        String SignName = sysParamsService.getValue(Constant.SysMSMParam
            .ALIYUN_SMS_SIGN_NAME.getValue(), true);
        String TemplateCode = sysParamsService.getValue(Constant.SysMSMParam
            .ALIYUN_SMS_SMS_CODE_TEMPLATE_CODE.getValue(), true);
        try {
            SendSmsRequest sendSmsRequest = new SendSmsRequest()
                .setSignName(SignName)
                .setTemplateCode(TemplateCode)
                .setPhoneNumbers(phone)
                .setTemplateParam(String.format("{\"code\":\"%s\"}", VerificationCode));
            RuntimeOptions runtime = new RuntimeOptions();
            // 复制代码运行请自行打印 API 的返回值
            SendSmsResponse sendSmsResponse = client.sendSmsWithOptions(sendSmsRequest, runtime);
            log.info("发送短信响应的requestID: {}", sendSmsResponse.getBody().getRequestId());
        } catch (Exception e) {
            // 如果发送失败了退还这次发送数
            String todayCountKey = RedisKeys.getSMSTodayCountKey(phone);
            redisUtils.delete(todayCountKey);
            // 错误 message
            log.error(e.getMessage());
            throw new RenException(ErrorCode.SMS_SEND_FAILED);
        }
    }
}

/**

```

```
* 创建阿里云连接
* @return 返回连接对象
*/
private Client createClient(){
    String ACCESS_KEY_ID = sysParamsService.getValue(Constant.SysMSMParam
        .ALIYUN_SMS_ACCESS_KEY_ID.getValue(),true);
    String ACCESS_KEY_SECRET = sysParamsService.getValue(Constant.SysMSMParam
        .ALIYUN_SMS_ACCESS_KEY_SECRET.getValue(),true);
    try {
        Config config = new Config()
            .setAccessKeyId(ACCESS_KEY_ID)
            .setAccessKeySecret(ACCESS_KEY_SECRET);
        // 配置 Endpoint。中国站请使用dysmsapi.aliyuncs.com
        config.endpoint = "dysmsapi.aliyuncs.com";
        return new Client(config);
    }catch (Exception e){
        // 错误 message
        log.error(e.getMessage());
        throw new RenException(ErrorCode.SMS_CONNECTION_FAILED);
    }
}

}

// ===== 文件: xiaozhi/modules/llm/service/LLMService.java =====
package xiaozhi.modules.llm.service;

/**
 * LLM服务接口
 * 支持多种大模型调用
 */
public interface LLMService {

    /**
     * 生成聊天记录总结
     *
     * @param conversation 对话内容
     * @param promptTemplate 提示词模板
     * @return 总结结果
     */
    String generateSummary(String conversation, String promptTemplate);

    /**
     * 生成聊天记录总结（使用默认提示词）
     *
     * @param conversation 对话内容
     * @return 总结结果
     */
    String generateSummary(String conversation);

    /**
     * 生成聊天记录总结（指定模型ID）
     *
     * @param conversation 对话内容
     * @param modelId 模型ID
     * @return 总结结果
     */
    String generateSummaryWithModel(String conversation, String modelId);

    /**
     * 生成聊天记录总结（指定模型ID和提示词模板）
     *

```

```
* @param conversation 对话内容
* @param promptTemplate 提示词模板
* @param modelId 模型ID
* @return 总结结果
*/
String generateSummary(String conversation, String promptTemplate, String modelId);

/**
 * 生成聊天记录总结（包含历史记忆合并）
 *
 * @param conversation 对话内容
 * @param historyMemory 历史记忆
 * @param promptTemplate 提示词模板
 * @param modelId 模型ID
 * @return 总结结果
 */
String generateSummaryWithHistory(String conversation, String historyMemory, String promptTemplate,
String modelId);

/**
 * 检查服务是否可用
 *
 * @return 是否可用
 */
boolean isAvailable();

/**
 * 检查指定模型的服务是否可用
 *
 * @param modelId 模型ID
 * @return 是否可用
 */
boolean isAvailable(String modelId);

/**
 * 生成会话标题
 *
 * @param conversation 对话内容
 * @param modelId 模型ID
 * @return 标题（约15字）
 */
String generateTitle(String conversation, String modelId);
}

// ===== 文件: xiaozhi/modules/llm/service/impl/OpenAISTyleLLMServiceImpl.java =====
package xiaozhi.modules.llm.service.impl;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
```

```

import cn.hutool.json.JSONArray;
import cn.hutool.json.JSONObject;
import cn.hutool.json.JSONUtil;
import lombok.extern.slf4j.Slf4j;
import xiaozhi.modules.llm.service.LLMService;
import xiaozhi.modules.model.entity.ModelConfigEntity;
import xiaozhi.modules.model.service.ModelConfigService;

/**
 * OpenAI风格API的LLM服务实现
 * 支持阿里云、DeepSeek、ChatGLM等兼容OpenAI API的模型
 */
@Slf4j
@Service
public class OpenAIStyleLLMServiceImpl implements LLMService {

    // 需要禁用思考模式的平台域名及其对应参数
    private static final Map<String, Map<String, Object>> THINKING_DISABLED_DOMAINS = new LinkedHashMap<>();
    static {
        THINKING_DISABLED_DOMAINS.put("aliyuncs.com", Map.of("enable_thinking", false));
        Map<String, Object> thinkingDisabled = Map.of("thinking", Map.of("type", "disabled"));
        THINKING_DISABLED_DOMAINS.put("bigmodel.cn", thinkingDisabled);
        THINKING_DISABLED_DOMAINS.put("moonshot.cn", thinkingDisabled);
        THINKING_DISABLED_DOMAINS.put("volces.com", thinkingDisabled);
    }

    @Autowired
    private ModelConfigService modelConfigService;

    private final RestTemplate restTemplate = new RestTemplate();

    /**
     * 根据域名自动禁用思考模式
     */
    private void applyThinkingDisabled(String baseUrl, Map<String, Object> requestBody) {
        for (Map.Entry<String, Map<String, Object>> entry : THINKING_DISABLED_DOMAINS.entrySet()) {
            if (baseUrl.contains(entry.getKey())) {
                requestBody.putAll(entry.getValue());
                log.info("为域名 {} 禁用思考模式, 参数: {}", baseUrl, entry.getValue());
                break;
            }
        }
    }

    private static final String DEFAULT_SUMMARY_PROMPT = "你是一个经验丰富的记忆总结者, 擅长将对话内容进行总结摘要, 遵循以下规则: \n1、总结用户的重要信息, 以便在未来的对话中提供更个性化的服务\n2、不要重复总结, 不要遗忘之前记忆, 除非原来的记忆超过了1800字, 否则不要遗忘、不要压缩用户的历史记忆\n3、用户操控的设备音量、播放音乐、天气、退出、不想对话等和用户本身无关的内容, 这些信息不需要加入到总结中\n4、聊天内容中的今天的日期时间、今天的天气情况与用户事件无关的数据, 这些信息如果当成记忆存储会影响后续对话, 这些信息不需要加入到总结中\n5、不要把设备操控的成果结果和失败结果加入到总结中, 也不要用户的一些废话加入到总结中\n6、不要为了总结而总结, 如果用户的聊天没有意义, 请返回原来的历史记录也是可以的\n7、只需要返回总结摘要, 严格控制在1800字内\n8、不要包含代码、xml, 不需要解释、注释和说明, 保存记忆时仅从对话提取信息, 不要混入示例内容\n9、如果提供了历史记忆, 请将新对话内容与历史记忆进行智能合并, 保留有价值的历史信息, 同时添加新的重要信息\n\n历史记忆: \n{history_memory}\n\n新对话内容: \n{conversation}";

    private static final String DEFAULT_TITLE_PROMPT = "请根据以下对话内容, 生成一个简洁的会话标题(约15字以内), 只返回标题, 不要包含任何解释或标点符号: \n{conversation}";

```

```
@Override
public String generateSummary(String conversation) {
    return generateSummary(conversation, null, null);
}

@Override
public String generateSummaryWithModel(String conversation, String modelId) {
    return generateSummary(conversation, null, modelId);
}

@Override
public String generateSummary(String conversation, String promptTemplate, String modelId) {
    if (!isAvailable()) {
        log.warn("LLM服务不可用，无法生成总结");
        return "LLM服务不可用，无法生成总结";
    }

    try {
        // 从智控台获取LLM模型配置
        ModelConfigEntity llmConfig;
        if (modelId != null && !modelId.trim().isEmpty()) {
            // 通过具体模型ID获取配置
            llmConfig = modelConfigService.getModelByIdFromCache(modelId);
        } else {
            // 保持向后兼容，使用默认配置
            llmConfig = getDefaultLLMConfig();
        }

        if (llmConfig == null || llmConfig.getConfigJson() == null) {
            log.error("未找到可用的LLM模型配置，modelId: {}", modelId);
            return "未找到可用的LLM模型配置";
        }

        JSONObject configJson = llmConfig.getConfigJson();
        String baseUrl = configJson.getStr("base_url");
        String model = configJson.getStr("model_name");
        String apiKey = configJson.getStr("api_key");
        Double temperature = configJson.getDouble("temperature");
        Integer maxTokens = configJson.getInt("max_tokens");

        if (StringUtils.isBlank(baseUrl) || StringUtils.isBlank(apiKey)) {
            log.error("LLM配置不完整，baseUrl或apiKey为空");
            return "LLM配置不完整，无法生成总结";
        }

        // 构建提示词
        String prompt = (promptTemplate != null ? promptTemplate : DEFAULT_SUMMARY_PROMPT).replace(
            "{conversation}",
            conversation);

        // 构建请求体
        Map<String, Object> requestBody = new HashMap<>();
        requestBody.put("model", model != null ? model : "gpt-3.5-turbo");

        Map<String, Object> message = new HashMap<>();
        message.put("role", "user");
        message.put("content", prompt);

        requestBody.put("messages", List.of(message));
    }
}
```

```

requestBody.put("temperature", temperature != null ? temperature : 0.7);
requestBody.put("max_tokens", maxTokens != null ? maxTokens : 2000);

// 禁用思考模式
applyThinkingDisabled(baseUrl, requestBody);

// 发送HTTP请求
HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_JSON);
headers.set("Authorization", "Bearer " + apiKey);

HttpEntity<Map<String, Object>> entity = new HttpEntity<>(requestBody, headers);

// 构建完整的API URL
String apiUrl = baseUrl;
if (!apiUrl.endsWith("/chat/completions")) {
    if (!apiUrl.endsWith("/")) {
        apiUrl += "/";
    }
    apiUrl += "chat/completions";
}

ResponseEntity<String> response = restTemplate.exchange(
    apiUrl, HttpMethod.POST, entity, String.class);

if (response.getStatusCode().is2xxSuccessful()) {
    JSONObject responseJson = JSONUtil.parseObj(response.getBody());
    JSONArray choices = responseJson.getJSONArray("choices");
    if (choices != null && choices.size() > 0) {
        JSONObject choice = choices.getJSONObject(0);
        JSONObject messageObj = choice.getJSONObject("message");
        return messageObj.getStr("content");
    }
} else {
    log.error("LLM API调用失败, 状态码: {}, 响应: {}", response.getStatusCode(), response.ge
tBody());
}
} catch (Exception e) {
    log.error("调用LLM服务生成总结时发生异常, modelId: {}", modelId, e);
}

return "生成总结失败, 请稍后重试";
}

@Override
public String generateSummary(String conversation, String promptTemplate) {
    return generateSummary(conversation, promptTemplate, null);
}

@Override
public String generateSummaryWithHistory(String conversation, String historyMemory, String promptTem
plate,
    String modelId) {
    if (!isAvailable()) {
        log.warn("LLM服务不可用, 无法生成总结");
        return "LLM服务不可用, 无法生成总结";
    }

    try {
        // 从智控台获取LLM模型配置

```

```
ModelConfigEntity llmConfig;
if (modelId != null && !modelId.trim().isEmpty()) {
    llmConfig = modelConfigService.getModelByIdFromCache(modelId);
} else {
    llmConfig = getDefaultLLMConfig();
}

if (llmConfig == null || llmConfig.getConfigJson() == null) {
    log.error("未找到可用的LLM模型配置, modelId: {}", modelId);
    return "未找到可用的LLM模型配置";
}

JSONObject configJson = llmConfig.getConfigJson();
String baseUrl = configJson.getStr("base_url");
String model = configJson.getStr("model_name");
String apiKey = configJson.getStr("api_key");

if (StringUtils.isBlank(baseUrl) || StringUtils.isBlank(apiKey)) {
    log.error("LLM配置不完整, baseUrl或apiKey为空");
    return "LLM配置不完整, 无法生成总结";
}

// 构建提示词, 包含历史记忆
String prompt = (promptTemplate != null ? promptTemplate : DEFAULT_SUMMARY_PROMPT)
    .replace("{history_memory}", historyMemory != null ? historyMemory : "无历史记忆")
    .replace("{conversation}", conversation);

// 构建请求体
Map<String, Object> requestBody = new HashMap<>();
requestBody.put("model", model != null ? model : "gpt-3.5-turbo");

Map<String, Object> message = new HashMap<>();
message.put("role", "user");
message.put("content", prompt);

requestBody.put("messages", List.of(message));
requestBody.put("temperature", 0.2);
requestBody.put("max_tokens", 2000);

// 禁用思考模式
applyThinkingDisabled(baseUrl, requestBody);

// 发送HTTP请求
HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_JSON);
headers.set("Authorization", "Bearer " + apiKey);

HttpEntity<Map<String, Object>> entity = new HttpEntity<>(requestBody, headers);

// 构建完整的API URL
String apiUrl = baseUrl;
if (!apiUrl.endsWith("/chat/completions")) {
    if (!apiUrl.endsWith("/")) {
        apiUrl += "/";
    }
    apiUrl += "chat/completions";
}

ResponseEntity<String> response = restTemplate.exchange(
    apiUrl, HttpMethod.POST, entity, String.class);
```

```

        if (response.getStatusCode().is2xxSuccessful()) {
            JSONObject responseJson = JSONUtil.parseObj(response.getBody());
            JSONArray choices = responseJson.getJSONArray("choices");
            if (choices != null && choices.size() > 0) {
                JSONObject choice = choices.getJSONObject(0);
                JSONObject messageObj = choice.getJSONObject("message");
                return messageObj.getStr("content");
            }
        } else {
            log.error("LLM API调用失败, 状态码: {}, 响应: {}", response.getStatusCode(), response.ge
tBody());
        }
    } catch (Exception e) {
        log.error("调用LLM服务生成总结时发生异常, modelId: {}", modelId, e);
    }

    return "生成总结失败, 请稍后重试";
}

@Override
public boolean isAvailable() {
    try {
        ModelConfigEntity defaultLLMConfig = getDefaultLLMConfig();
        if (defaultLLMConfig == null || defaultLLMConfig.getConfigJson() == null) {
            return false;
        }

        JSONObject configJson = defaultLLMConfig.getConfigJson();
        String baseUrl = configJson.getStr("base_url");
        String apiKey = configJson.getStr("api_key");

        return baseUrl != null && !baseUrl.trim().isEmpty() &&
            apiKey != null && !apiKey.trim().isEmpty();
    } catch (Exception e) {
        log.error("检查LLM服务可用性时发生异常: ", e);
        return false;
    }
}

@Override
public boolean isAvailable(String modelId) {
    try {
        if (modelId == null || modelId.trim().isEmpty()) {
            return isAvailable();
        }

        // 通过具体模型ID获取配置
        ModelConfigEntity modelConfig = modelConfigService.getModelByIdFromCache(modelId);
        if (modelConfig == null || modelConfig.getConfigJson() == null) {
            log.warn("未找到指定的LLM模型配置, modelId: {}", modelId);
            return false;
        }

        JSONObject configJson = modelConfig.getConfigJson();
        String baseUrl = configJson.getStr("base_url");
        String apiKey = configJson.getStr("api_key");

        return baseUrl != null && !baseUrl.trim().isEmpty() &&
            apiKey != null && !apiKey.trim().isEmpty();
    }
}

```

```
    } catch (Exception e) {
        log.error("检查LLM服务可用性时发生异常, modelId: {}", modelId, e);
        return false;
    }
}

/**
 * 从智控台获取默认的LLM模型配置
 */
private ModelConfigEntity getDefaultLLMConfig() {
    try {
        // 获取所有启用的LLM模型配置
        List<ModelConfigEntity> llmConfigs = modelConfigService.getEnabledModelsByType("LLM");
        if (llmConfigs == null || llmConfigs.isEmpty()) {
            return null;
        }

        // 优先返回默认配置, 如果没有默认配置则返回第一个启用的配置
        for (ModelConfigEntity config : llmConfigs) {
            if (config.getIsDefault() != null && config.getIsDefault() == 1) {
                return config;
            }
        }

        return llmConfigs.get(0);
    } catch (Exception e) {
        log.error("获取LLM模型配置时发生异常: ", e);
        return null;
    }
}

@Override
public String generateTitle(String conversation, String modelId) {
    if (!isAvailable()) {
        log.warn("LLM服务不可用, 无法生成标题");
        return null;
    }

    try {
        ModelConfigEntity llmConfig;
        if (modelId != null && !modelId.trim().isEmpty()) {
            llmConfig = modelConfigService.getModelByIdFromCache(modelId);
        } else {
            llmConfig = getDefaultLLMConfig();
        }

        if (llmConfig == null || llmConfig.getConfigJson() == null) {
            log.error("未找到可用的LLM模型配置, modelId: {}", modelId);
            return null;
        }

        JSONObject configJson = llmConfig.getConfigJson();
        String baseUrl = configJson.getStr("base_url");
        String model = configJson.getStr("model_name");
        String apiKey = configJson.getStr("api_key");

        if (StringUtils.isBlank(baseUrl) || StringUtils.isBlank(apiKey)) {
            log.error("LLM配置不完整, baseUrl或apiKey为空");
            return null;
        }
    }
}
```

```

String prompt = DEFAULT_TITLE_PROMPT.replace("{conversation}", conversation);

Map<String, Object> requestBody = new HashMap<>();
requestBody.put("model", model != null ? model : "gpt-3.5-turbo");

Map<String, Object> message = new HashMap<>();
message.put("role", "user");
message.put("content", prompt);

requestBody.put("messages", List.of(message));
requestBody.put("temperature", 0.3);
requestBody.put("max_tokens", 50);

// 禁用思考模式
applyThinkingDisabled(baseUrl, requestBody);

HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_JSON);
headers.set("Authorization", "Bearer " + apiKey);

HttpEntity<Map<String, Object>> entity = new HttpEntity<>(requestBody, headers);

String apiUrl = baseUrl;
if (!apiUrl.endsWith("/chat/completions")) {
    if (!apiUrl.endsWith("/")) {
        apiUrl += "/";
    }
    apiUrl += "chat/completions";
}

ResponseEntity<String> response = restTemplate.exchange(
    apiUrl, HttpMethod.POST, entity, String.class);

if (response.getStatusCode().is2xxSuccessful()) {
    JSONObject responseJson = JSONUtil.parseObj(response.getBody());
    JSONArray choices = responseJson.getJSONArray("choices");
    if (choices != null && choices.size() > 0) {
        JSONObject choice = choices.getJSONObject(0);
        JSONObject messageObj = choice.getJSONObject("message");
        String title = messageObj.getStr("content");
        if (StringUtils.isNotBlank(title)) {
            title = title.trim().replaceAll("[,。！？\\、：；'\"\\\" 【】 ()]", "");
            if (title.length() > 15) {
                title = title.substring(0, 15);
            }
            return title;
        }
    }
} else {
    log.error("LLM API调用失败, 状态码: {}, 响应: {}", response.getStatusCode(), response.get
tBody());
}
} catch (Exception e) {
    log.error("调用LLM服务生成标题时发生异常, modelId: {}", modelId, e);
}

return null;
}
}

```